CS8602-COMPILER DESIGN

III Year / VI Sem

Department of CSE
Panimalar Engineering College
Chennai-123

OBJECTIVES

- To learn the various phases of compiler.
- To learn the various parsing techniques.
- To understand intermediate code generation and run-time environment.
- To learn to implement front-end of the compiler.
- To learn to implement code generator.

UNIT I INTRODUCTION TO COMPILERS 9

Structure of a compiler – Lexical Analysis – Role of Lexical Analyzer – Input Buffering – Specification of Tokens – Recognition of Tokens – Lex – Finite Automata – Regular Expressions to Automata – Minimizing DFA.

UNIT II SYNTAX ANALYSIS 12

Role of Parser – Grammars – Error Handling – Context-free grammars – Writing a grammar – Top Down Parsing – General Strategies Recursive Descent Parser Predictive Parser-LL(1) Parser-Shift Reduce Parser-LR Parser-LR (0) Item Construction of SLR Parsing Table -Introduction to LALR Parser – Error Handling and Recovery in Syntax Analyzer-YACC.

UNIT III INTERMEDIATE CODE GENERATION 8

Syntax Directed Definitions, Evaluation Orders for Syntax Directed Definitions, Intermediate Languages: Syntax Tree, Three Address Code, Types and Declarations, Translation of Expressions, Type Checking.

UNIT IV RUN-TIME ENVIRONMENT AND CODE GENERATION 8

Storage Organization, Stack Allocation Space, Access to Non-local Data on the Stack, Heap Management – Issues in Code Generation – Design of a simple Code Generator.

UNIT V CODE OPTIMIZATION 8

Principal Sources of Optimization – Peep-hole optimization – DAG-Optimization of Basic Blocks-Global Data Flow Analysis – Efficient Data Flow Algorithm.

LIST OF EXPERIMENTS:

1. Develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, constants, comments, operators etc.). Create a symbol table, while recognizing identifiers.
2. Implement a Lexical Analyzer using Lex Tool
3. Implement an Arithmetic Calculator using LEX and YACC
4. Generate three address code for a simple program using LEX and YACC.
5. Implement simple code optimization techniques (Constant folding, Strength reduction and Algebraic transformation)
6. Implement back-end of the compiler for which the three address code is given as input and the 8086 assembly language code is produced as output.

UNIT-I
INTRODUCTION TO COMPILER

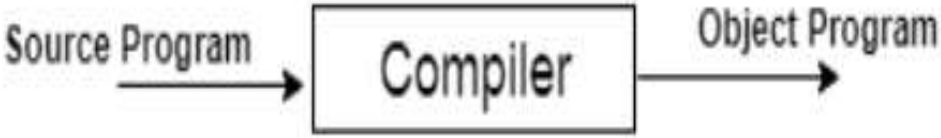
- A compiler is a translator that converts the high-level language into the machine language.
- High-level language →developer
- Machine language → processor.
- Compiler is used to show errors to the programmer.
- The **main purpose of compiler** is to change the code written in one language without changing the meaning of the program.

UNIT-I
INTRODUCTION TO COMPILER

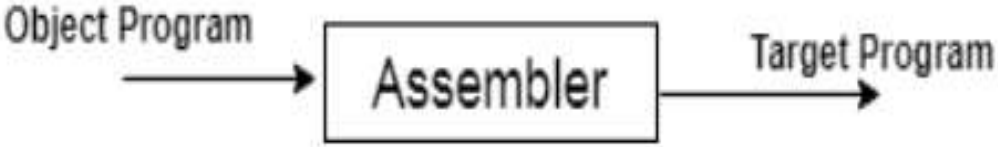
- Makes the end code efficient which is optimized for execution time and memory space.
- Compiling process includes basic translation mechanisms and error detection.
- Compiler process goes through lexical, syntax, and semantic analysis at the front end, and code generation and optimization at a back-end.

Execution process of source program in Compiler

- It executes into two parts.
- In the first part, the source program compiled and translated into the object program (low level language).



- In the second part, object program translated into the target program through the assembler.



TRANSLATORS-COMPILATION AND INTERPRETATION

TRANSLATOR

- A translator is a program that takes as input a program written in one language and produces as output a program in another language.
- Translator performs another very important role, the error-detection.
- Any violation of HLL(High Level Language) specification would be detected and reported to the programmers.

Types of Translators:

- Interpreter
- Assembler
- Compiler

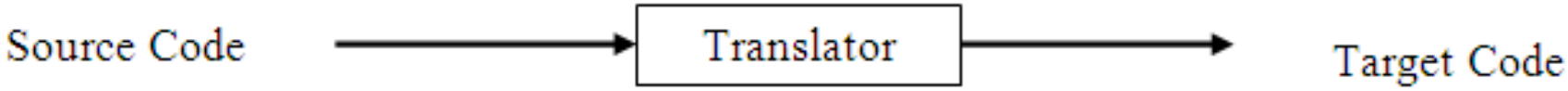
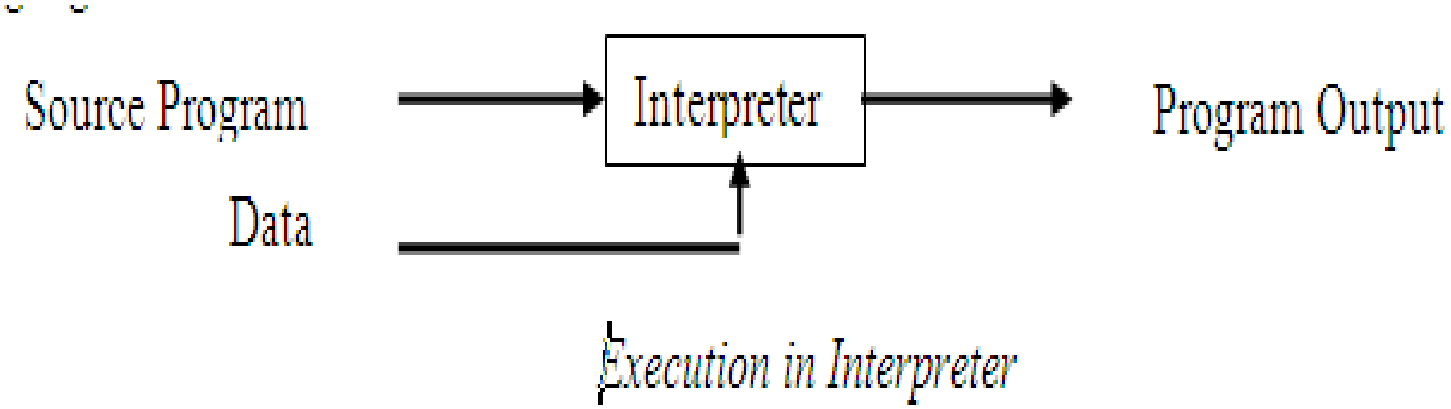


Fig1.1.1: Execution in Translator

INTERPRETER

- Is a program that appears to execute a source program as if it were machine language.
- It is one of the translators that translate high level language to low level language



INTERPRETER

- During execution, it checks line by line for errors. Languages such as BASIC, SNOBOL, LISP can be translated using interpreters.
- JAVA also uses interpreter.
- The process of interpretation- can be carried out in following phases.
- 1. Lexical analysis
- 2. Syntax analysis
- 3. Semantic analysis
- 4. Direct Execution
- **Example:** BASIC , Lower Version of Pascal, SNOBOL, LISP & JAVA
-

INTERPRETER

Advantages

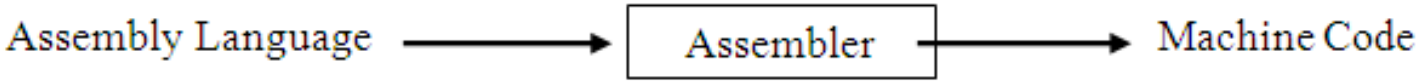
- Modification of user program can be easily made and implemented as execution proceeds.
- Type of object that denotes various may change dynamically.
- Debugging a program and finding errors is simplified task for a program used for interpretation.
- The interpreter for the language makes it machine independent.

Disadvantages

- The execution of the program is slower.
- Memory consumption is more.

ASSEMBLER

- Programmers found it difficult to write or read programs in machine language.
- They begin to use a mnemonic (symbols) for each machine instruction, which they would subsequently translate into machine language.
- Such a mnemonic machine language is now called an assembly language.
- Assembler -- assembly language in to machine language.
- The input to an assembler program is called source program, the output is a machine language translation object program.
- It translates assembly level language to machine code.



Execution in Assembler

ASSEMBLER

Advantages:

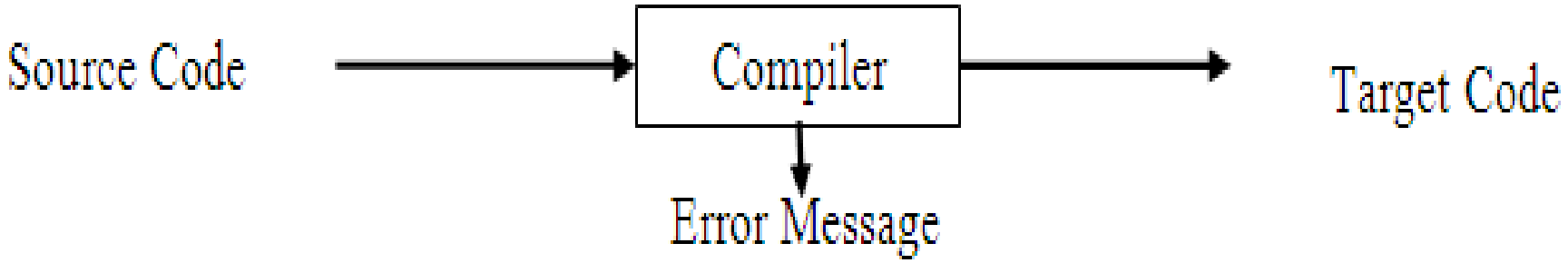
- Debugging and verifying
- Making compilers->Understanding assembly coding techniques is necessary for making compilers, debuggers and other development tools.
- Optimizing code for size
- Optimizing code for speed.

Disadvantages:

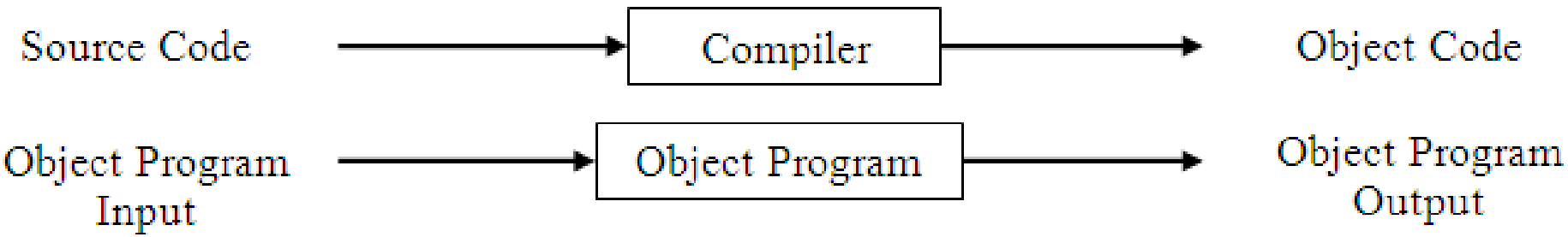
- Development time. Writing code in assembly language takes much longer than writing in a high-level language.
- Reliability and security. It is easy to make errors in assembly code.
- Debugging and verifying. Assembly code is more difficult to debug and verify because there are more possibilities for errors than in high-level code.
- Portability. Assembly code is platform-specific.

COMPILER

- Compiler is a translator program
- Translates a program written in (HLL) the source program and translate it into an equivalent program in (MLL) the target program.
- As an ***important role of a compiler*** is error showing to the programmer.
- Executing a program written in HLL programming language is basically of two parts.
- 1. The source program must first be compiled translated into a object program.
- 2.Then the results object program is loaded into a memory executed.
- **Example:** C, C++, COBOL, higher version of Pascal.



Execution in Compiler



Execution Process of Source Program in Compiler

List of Compilers

- Ada compilers
- ALGOL compilers
- BASIC compilers
- C# compilers
- C compilers
- C++ compilers
- COBOL compilers
- D compilers
- Common Lisp compilers
- Fortran compilers
- Java compilers
- Pascal compilers
- PL/I compilers
- Python compilers
- Smalltalk compilers

Types of Compilers

1.Traditional Compiler (C,C++ & Pascal)

Convert source program (HLL) into its equivalent in native machine code or object code.

2.Interpreters(LISP, SNOBOL & Java1.0)

These Compilers first convert source code into intermediate code and then interprets(emulates) it to its equivalent machine code.

3.Cross Compilers

These are the compilers that run on one machine and produce code for another machine.

4.Incremental Compilers

Separate the source into user defined steps; compiling / recompiling step-by-step; interpreting steps in a given order.

5.Converters(COBOL to C++)

These programs will be compiling from one HLL to another

6.Just – In Time (JIT) Compilers (Java, Microsoft.NET)

Runtime compilers from intermediate language (byte code, MSIL) to executable code / native machine code.

7.Ahead of Time(AoT) Compiler (.NET ngen)

Precompilers to the native code for Java & .NET

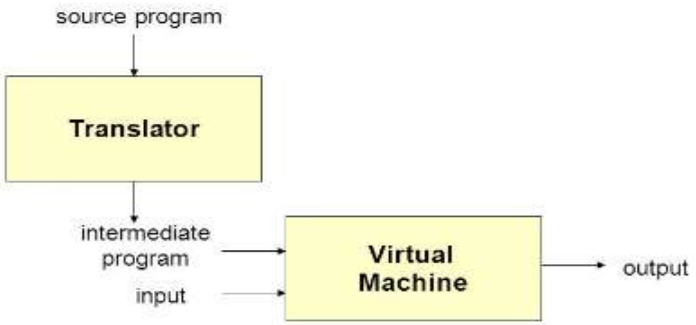
8.Binary Compilation

Compiling object code of one platform into object code of another platform.

Hybrid Compiler

- Java language processors combine compilation and interpretation

- A hybrid compiler, e.g. Java



- First -- compiled into an intermediate form called byte codes.
- The byte codes are then interpreted by a virtual machine.
- A benefit of this arrangement is that byte codes compiled on one machine can be interpreted on another machine, perhaps across a network.
- In order to achieve faster processing of inputs to outputs, some Java compilers, called just-in-time compilers, translate the byte codes into machine language immediately before they run the intermediate program to process the input.

Sl.No	Compiler	Interpreter
1	Compiler works on the complete program at once. It takes the entire program as input.	Interpreter program works line-by-line. It takes one statement at a time as input.
2	Compiler generates intermediate code, called the object code or machine code.	Interpreter does not generate intermediate object code or machine code .
3	Compiler executes conditional control statements (like if-else and switch-case) and logical constructs faster than interpreter.	Interpreter executes conditional control statements at a much slower speed.
4	Compiled programs take more memory because the entire object code has to reside in memory	Interpreter does not generate intermediate object code. As a result, interpreted programs are more memory efficient.
5	Compile once and run anytime. Compiled program does not need to be compiled every time.	Interpreted programs are interpreted line-by-line every time they are run.

Sl.No	Compiler	Interpreter
6	Errors are reported after the entire program is checked for syntactical and other errors.	Error is reported as soon as the first error is encountered. Rest of the program will not be checked until the existing error is removed.
7	A compiled language is more difficult to debug.	Debugging is easy because interpreter stops and reports errors as it encounters them.
8	Compiler does not allow a program to run until it is completely error-free.	Interpreter runs the program from first line and stops execution only if it encounters an error.
9	Compiled languages are more efficient but difficult to debug.	Interpreted languages are less efficient but easier to debug.
10	C, C++, COBOL	BASIC, Visual Basic, Python, Ruby, PHP, Perl, MATLAB, Lisp

Features of Compilers

- Correctness
- Speed of compilation
- Preserve the correct the meaning of the code
- The speed of the target code
- Recognize legal and illegal program constructs
- Good error reporting/handling
- Code debugging help

Types of Compiler

- Single Pass Compilers
- Two Pass Compilers
- Multipass Compilers

Single Pass Compiler

- In single pass Compiler source code directly transforms into machine code.
- **Pascal language.**

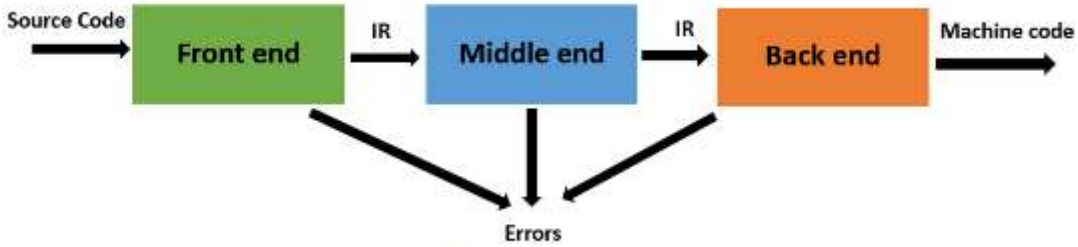
Two Pass Compiler

- Two pass Compiler is divided into two sections.
- **Front end:**
- It maps legal code into Intermediate Representation (IR).
- **Back end:**
- It maps IR onto the target machine
- The Two pass compiler method also simplifies the retargeting process. It also allows multiple front ends.

Two Pass Compiler

Multipass Compilers

- Multipass compiler processes the source code or syntax tree of a program several times.
- It divided a large program into multiple small programs and process them.
- It develops multiple intermediate codes.
- All of these multipass take the output of the previous phase as an input.
- So it requires less memory.
- It is also known as 'Wide Compiler'.



Multipass Compilers

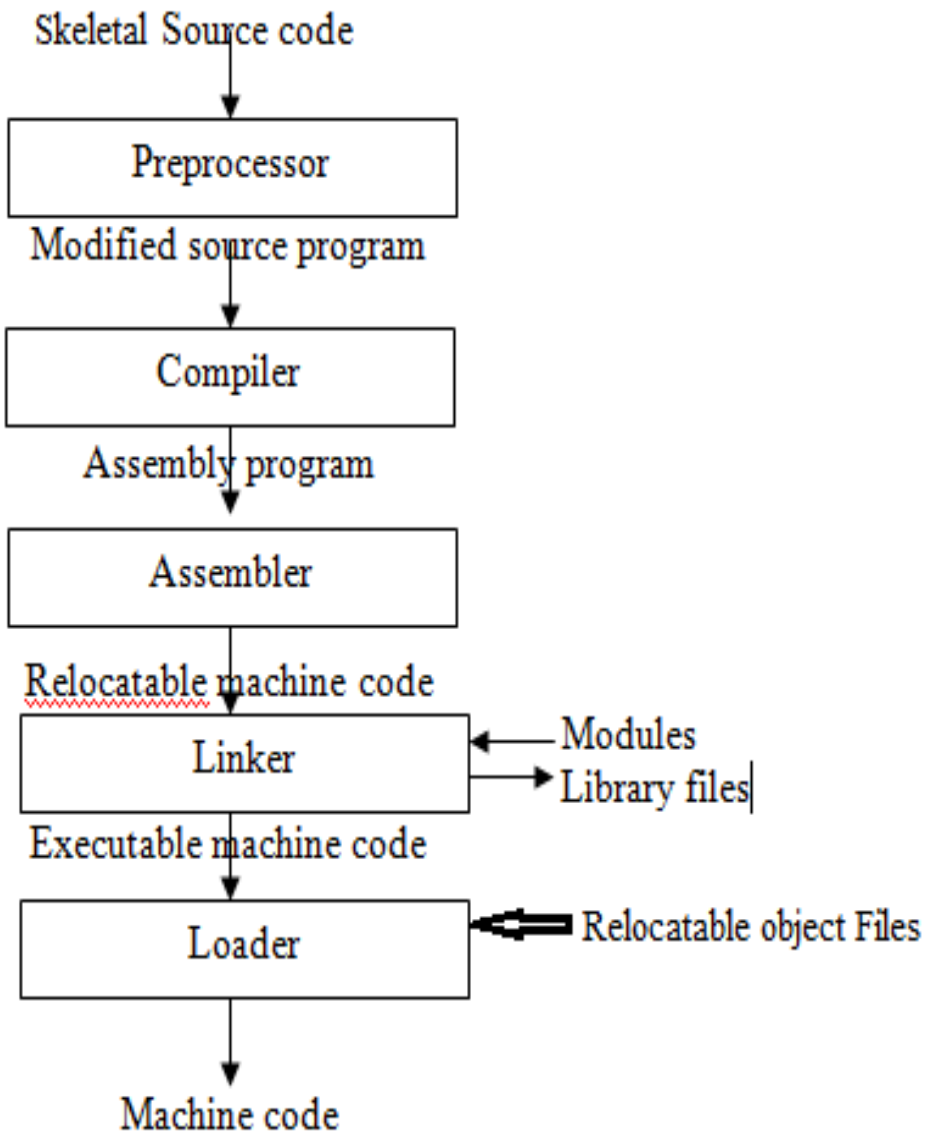
Tasks of Compiler

- Breaks up the source program into pieces and impose grammatical structure on them
- Allows you to construct the desired target program from the intermediate representation and also create the symbol table
- Compiles source code and detects errors in it
- Manage storage of all variables and codes.
- Support for separate compilation
- Read, analyze the entire program, and translate to semantically equivalent
- Translating the source code into object code depending upon the type of machine

History of Compiler

- The "compiler" -1950s by Grace Murray Hopper
- The first compiler - John Backum and his group between 1954 and 1957 at IBM
- COBOL was the first programming language which was compiled on multiple platforms in 1960
- The study of the scanning and parsing issues were pursued in the 1960s and 1970s to provide a complete solution

LANGUAGE PROCESSING SYSTEM (COUSINS OF COMPILER)



1.Preprocessor

- Takes the source program as input and produces an extended version of it.
- Extending the Macros and including header files etc.
- A preprocessor produce input to compilers.
- **<u>Functions:</u>**
- ***Macro processing***
- ***File inclusion***
- ***Rational preprocessor***
- ***Language Extensions***

- **2.Compiler:**
- Translator high level language into machine language.
- Additionally reports to its client the blunders of errors in the source program.
- Help the user in rectifying the errors, and execute the code.
- **3 Assembler:**
- Program – converts assembly language program into its equivalent machine language code

4 Linker

- Takes as input are locatable code and gathers the library capacities, relocatable item documents, and delivers its machine code.
- The major task :
  - To search and locate referenced module/routines in a program and to determine the memory location where these codes will be loaded

5.Loader

- Part of operating system and is responsible for loading executable files into memory and execute them.
- It calculates the size of a program (instruction & data) and creates memory space for it.

Linking

- Permits us to make a single program from several files of relocatable machine code.
- These files may have been result of several different compilations, one or more may be library routines provided by the system available to any program that needs them.

Software Tools

- STRUCTURE EDITORS
- PRETTY PRINTERS
- STATIC CHECKERS
- INTERPRETERS

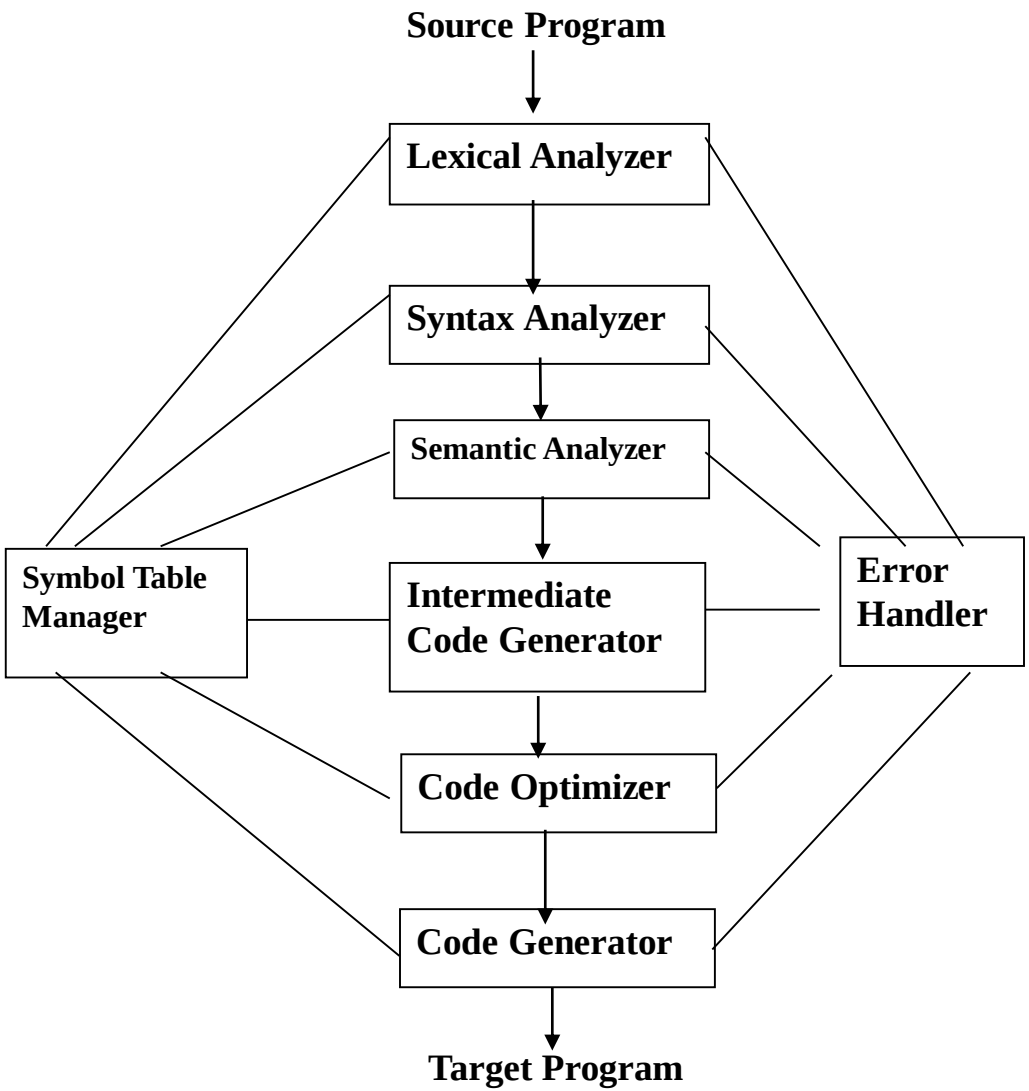
THE ANALYSIS-SYNTHESIS MODEL OF COMPILATION (Parts of compilation)

- Two parts : **analysis and synthesis**.
- **Analysis part** - breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program.
- **Synthesis part** - constructs the desired target program from the intermediate representation and the information in the symbol table.
- The **analysis part** is often called the front end of the compiler;
- The **synthesis part** is the back end..

THE PHASES OF A COMPILER

- **Compiler consists of 6 phases**.
- 1) **Lexical analysis** - it contains a sequence of characters called tokens. Input is source program & the output is tokens.
- 2) **Syntax analysis** - input is token and the output is parse tree
- 3) **Semantic analysis** - input is parse tree and the output is expanded version of parse tree
- 4) **Intermediate Code generation** - Here all the errors are checked & it produce an intermediate code.
- 5) **Code Optimization** - the intermediate code is optimized here to get the target program.
- 6) **Code Generation** - this is the final step & here the target program code is generated.
- First three phases, forms the **analysis portion** of a compiler
- Last three phases form the **synthesis portion** of a compiler
- Two other activities
  - Symbol-table management
  - Error handling

THE PHASES OF A COMPILER



1 Lexical Analysis | Linear |Scanner

- The first phase of a compiler
- Read S.P char by char -- returns the tokens (S.P) and groups char→ meaningful sequences (**lexemes)**
- output - each lexeme :
- ***<token-name, attribute-value>***
  - Passes to the syntax analysis.
- ***token-name*** → abstract symbol that is used during syntax analysis
- ***attribute-value*** →points to an entry in the symbol table for this token.
- Information from the symbol-table entry is needed for semantic analysis and code generation.

Example : **position: = initial + rate * 60**

would be grouped in to the following tokens

The ***identifier*** position : <id,1>

The ***assignment*** symbol =: <=>

The ***identifier*** initial : <id,2>

The ***plus*** sign+: <+>

The ***identifier*** rate: <id,3>

The ***multiplication*** sign: <*>

The ***number*** 60: <60>

The blanks are usually eliminated during lexical analysis

- *S.P:* *position = initial + rate * 60*
- **Inter.Form : <id,1> <=> <id, 2> <+> <id, 3> <*> <60>**
- **Note**: Regular Expression→ describe tokens
- DFA → implementation of a lexical analyzer

2 Syntax Analysis | Hierarchical Analysis | Parsing

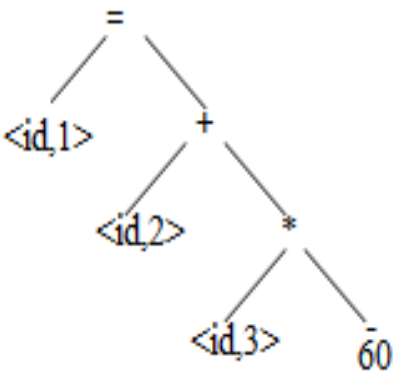


Fig 1.3.5 : Parse Tree

- second phase of the compiler
- I/p→ token stream (lexical analyzer)
- O/p→ syntax tree
- Representation →***syntax tree***
- interior node -- operation
- children node --- arguments of the operation.
- **<id,1> <=> <id, 2> <+> <id, 3> <*> <60>**
- Compiler use the grammatical structure to help analyze the source program and generate the target program.

2 Syntax Analysis

- **Note**: The syntax of a language is specified by a Context Free Grammar
- If it satisfies, the syntax analyzer creates a parse tree for the given program.
- We use Backus Naur Form to specify Contexts Free Grammar
- The hierarchical structure of a program is usually expressed by recursive rules. The **rules** are
  - Any identifier is an expression
  - Any number is an expression
  - If expression1 and expression2 are expressions, then so are
  - **expression$_1$ + expression$_2$**
  - **expression$_1$ * expression$_2$**
  - **(expression$_1$)**

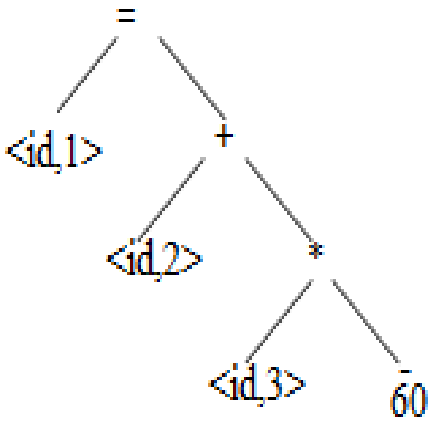


Fig 1.3.5 : Parse Tree

THE PHASES OF A COMPILER

3 SEMANTIC ANALYSIS

- Uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.
- Meaning of source string:
  - Matching of parenthesis
  - Matching if-else statement
  - Checking scope of operation
- An important part of semantic analysis is **type checking**,
- where the compiler checks that each operator has matching operands.

THE PHASES OF A COMPILER

3 SEMANTIC ANALYSIS

- **Coercions (Type casting/ Type Conversion)**
- An array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.
- **int arr[2.3];**
- The language specification may permit some type conversions called **coercions**.
- **For example**, a binary arithmetic operator.
- Suppose that position , initial , and rate have been declared to be floating-point numbers, and that the lexeme 60 by itself forms an integer.

THE PHASES OF A COMPILER
3 SEMANTIC ANALYSIS

- The type checker in the semantic analyzer.
- output of the semantic analyzer has an extra node for the operator **inttofloat.**
- Which explicitly converts its integer argument into a floating-point number.

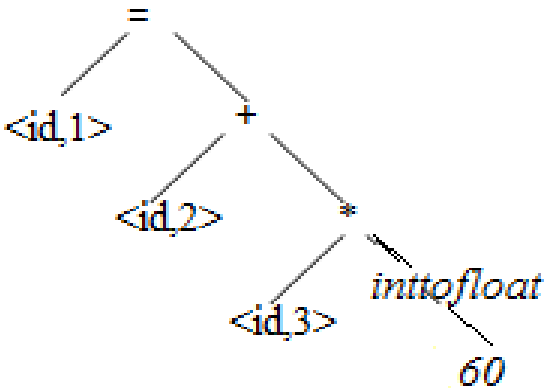


Fig 1.3.6: Semantic Analyzer

THE PHASES OF A COMPILER

4.INTERMEDIATE CODE GENERATION

- After syntax and semantic analysis, some compilers generate an explicit intermediate representation of the source program.
- This intermediate representation can have a variety of forms
  - Three Address Code
  - DAG
  - Syntax Tree
- Two properties
  - It should be easy to produce.
  - It should be easy to translate into the target program

THE PHASES OF A COMPILER

4.INTERMEDIATE CODE GENERATION

- **Three-address code properties**
  - Each three-address instruction has at most one operator in addition to the assignment.
  - The compiler must generate a temporary name to hold the value computed by each instruction.
  - Three-address instructions have few than 3 operands
- Example

temp1: = inttoreal (60)
temp2: = id3 * temp1
temp3: = id2 + temp2
id1: = temp3

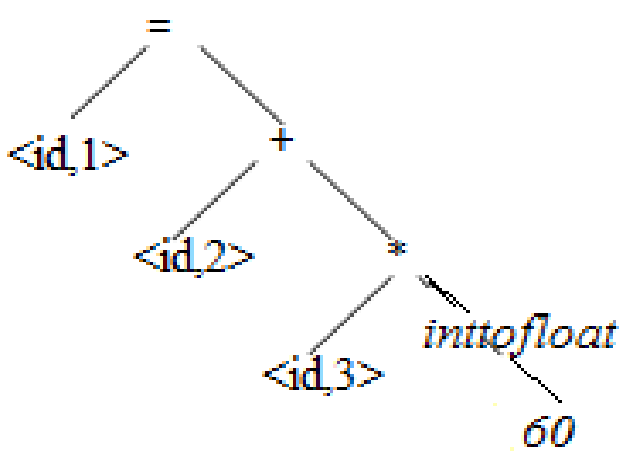


Fig 1.3.6: Semantic Analyzer

THE PHASES OF A COMPILER

5.Code Optimization

- The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.
- Better means faster---shorter code, or target code that consumes less power.
- During the code optimization, the result of the program is not affected.
- To improve the code generation, the optimization involves
  - deduction and removal of dead code (unreachable code)
  - calculation of constants in expressions and terms.
  - collapsing of repeated expression into temporary string.
  - loop unrolling.
  - moving code outside the loop.
  - removal of unwanted temporary variables.

THE PHASES OF A COMPILER

5.Code Optimization

- *I/p:*

t1 = inttofloat (60)

*t2 = id3 * t1*

t3 =id2 + t2

id1 = t3

- *O/P:*

*t1 = id3 * 60.0*

id1 =id2 + t1

- ***Optimizing compilers*** a significant amount of time is spent on this phase.
- There are simple optimizations that significantly improve the running time of the target program without slowing down compilation too much.

THE PHASES OF A COMPILER

6 Code Generation

- Input: Intermediate representation of the source program and maps it into the target language.
- If the target language is machine code, registers Or memory locations are selected for each of the variables used by the program.
- Intermediate instructions are translated into sequences of machine instructions that perform the same task.

Here, intermediate code translated into the machine code.

*t1 = id3 * 60.0*

id1 =id2 + t1

- LDF R2, id3
- MULF R2, R2, #60.0
- LDF Rl, id2
- ADDF Rl, Rl, R2
- STF idl, Rl
- The code generation involves
  - allocation of register and memory
  - generation of correct data types
  - generation of correct references
  - generation of missing code

Phases of Compiler

- Two supporting phases are:
  - Symbol Table Management
  - Error Detection and Reporting
- **Symbol Table Management:**
  - A symbol table is a data structure containing a record for each identifier with fields for the attributes of the identifier.
  - It allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly.

SYMBOL TABLE

1	position	...
2	initial	...
3	rate	...
4		

Phases of Compiler

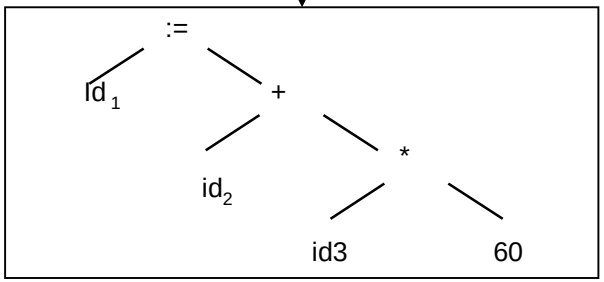
- **Error Detection and Reporting**
  - Each phase can encounter errors.
  - The syntax and semantic analysis phases handle a large fraction of the errors detectable by the compiler.
  - The lexical phase can detect errors where the character remaining in the input do not form any token of the language

position := initial + rate * 60

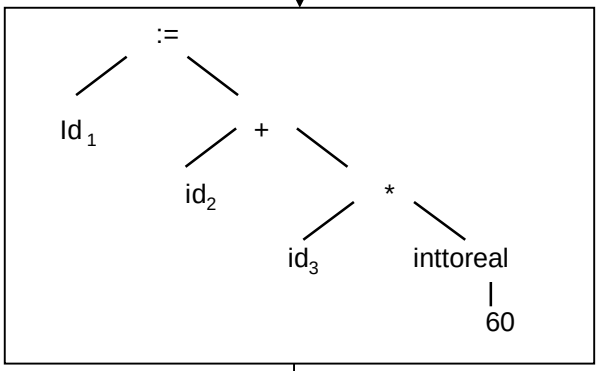
lexical analyzer

$id_1 := id_2 + id_3 * 60$

syntax analyzer



semantic analyzer



intermediate code generator

```
temp1 := inttoreal (60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1   := temp3
```

code optimizer

```
temp1 := id3 * 60.0
id1   := id2 + temp1
```

code generator

```
LDF    R2, id3
MULF   R2, R2, #60.0
LDF    Rl, id2
ADDF   Rl, Rl, R2
STF    idl, Rl
```

Exercises

c=a+b*d-4

c=(b+c)*(b+c)*2

$b=b^2$ -4ac

result=(height*width)+(rate*2)

CONSTRUCTION OF COMPILER TOOLS

- 1. ***Parser generators:***
  - that automatically produce syntax analyzers from input that is based on a context-free grammar.
- **2. *Scanner generators:***
  - produce lexical analyzers from a regular-expression description of the tokens of a language. The basic organization of the resulting lexical analyzer is in effect a finite automaton.
- **3. *Syntax-directed translation engines***
  - that produce collections of routines for walking a parse tree and generating intermediate code.
- **4. *Code-generator:***
  - generators that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.
- **5. *Data-flow analysis engines***
  - that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
- **6. *Compiler-construction toolkits***
  - that provide an integrated set of routines for constructing various phases of a compiler.

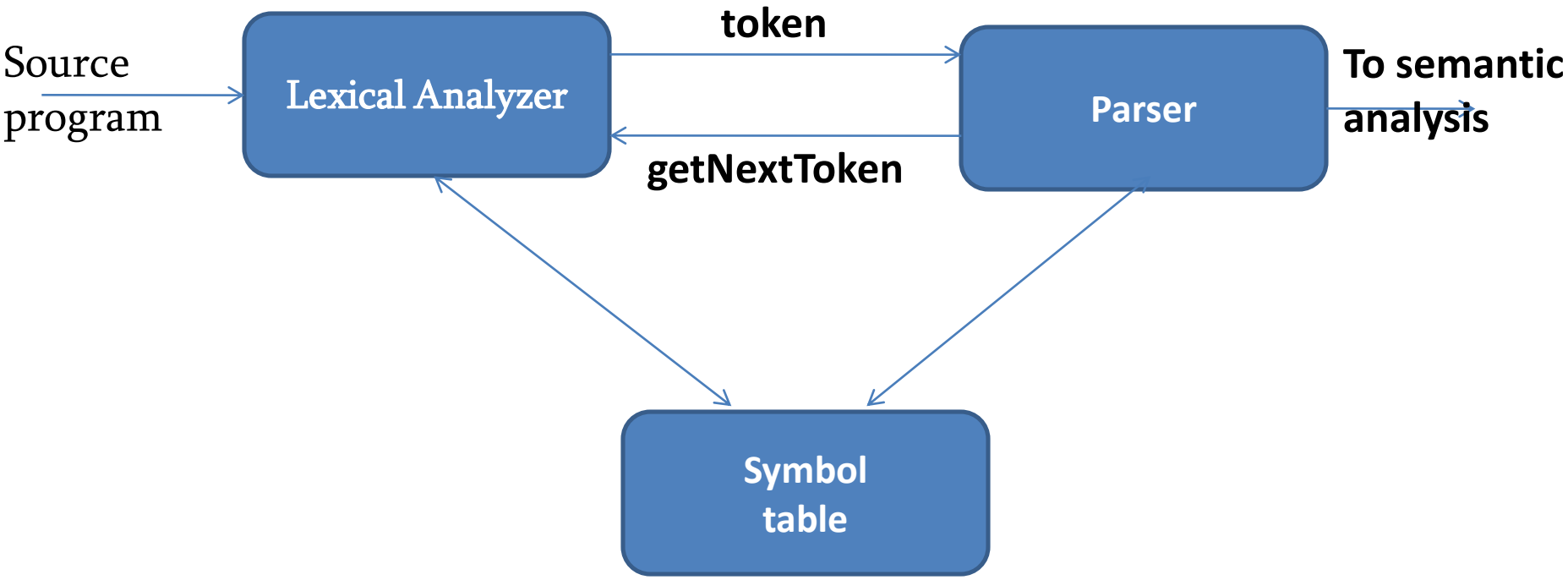
THE GROUPING OF PHASES

- The phases are collected into a front end and a back end.
- ***Front end***
  - The front end consists of those phases that depend primarily on the source language and are largely independent of the target machine. The phases are
    - Lexical analysis
    - Syntax analysis
    - Semantic analysis
    - Intermediate code generation
    - Some code optimization
- ***Back end***
  - The back end includes those portions of the compiler that depend on the target machine.
  - The phases in Back end are Code optimization phase, code generation phase along with error handling and symbol table operations

Lexical Analysis – Role of Lexical Analyzer

- First phase of a compiler / Scanner
- To identify the tokens : Regular expression
- Token recognizers: TD and FA
- **2.1 Main Task:**
- **Token Identification :**
  - To read input characters and produce output as a sequence of tokens that the parser uses for syntax analysis
- **getNextToken** :parser
  - Lexical analysis reads input characters until it can identify the next token.

The Role of Lexical Analyzer



- **<u>Secondary Task:</u>**
- Produces stream of tokens
- Stripping out from the comments and whitespaces while creating the tokens
- Generates symbol table:
  - Stores the information about identifiers, constants encountered in the input
- Keeps track of line numbers :
  - compare error with source file and line number mean while it reports the error encountered while generating the tokens.
- Macro preprocessor (e.g: #define pi 3.14)

- **<u>Lexical Analyzer are divided into two phases</u>:**
- 1 **.Scanning**
  - scans the source program to recognize the tokens
- 2. **Lexical analysis**
  - complex task, perform all secondary task.

- **2.2 Issues in Lexical analysis:**
- Simplicity of design
  - Separation of lexical from syntactical analysis
  - simplify at least one of the tasks
  - e.g. parser dealing with white spaces
- Improved compiler efficiency
  - Speedup reading input characters using specialized <u>buffering techniques</u>
- Enhanced compiler portability

- **2.3 Tokens, Patterns, Lexemes**
- **Token**
  - Sequence of character having a collective meaning.
- **Example:** keyword, identifier, operators, special character constants, etc
- **Pattern:** The set of rules by which set of string associated with single token
- **Example:**
  - keyword : character sequence forming that keyword
  - Identifiers
- **Lexeme:**
  - a sequence of characters in the source program matching a pattern for a token

Example

Token	Informal description	Sample lexemes
if	Characters i, f	if
else	Characters e, l, s, e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	Letter followed by letter and digits	pi, score, D2
number	Any numeric constant	3.14159, 0, 6.02e23
literal	Anything but " sorrounded by "	"core dumped"

```
printf("total = %d\n", score);
```

- **Example:** while(a >= 10)

Lexeme	Token
while	keyword
(	parenthesis
a	identifier
>=	relational operator
10	number
)	parenthesis

- # Attributes for Tokens
- When more than one lexeme can match a pattern, the lexical analyzer must provide the additional information about the particular lexeme that matched to the subsequent phase of the compiler.
- **Example 1 : PE = M * G * H**
- ◦<**id**, pointer to symbol table entry for **PE**>
- ◦<**assign_op**>
- ◦<**id**, pointer to symbol-table entry for **M**>
- ◦<**mult_op**>
- ◦<**id**, pointer to symbol-table entry for **G**>
- ◦<**mult_op**>
- ◦<**id**, pointer to symbol-table entry for **H**>
- **Symbol Table**

Symbol	Token	Data type	Initialized
E	id1	int	Yes
M	id2	int	Yes
C	id3	int	Yes

Lexical errors

- Some errors are out of power of lexical analyzer to recognize:
  - fi (a == f(x)) …
- However it may be able to recognize errors like:
  - d = 2r
- Such errors are recognized when no pattern for tokens matches a character sequence

Error recovery

- **Panic mode**:
  - successive characters are ignored until we reach to a well formed token
- **Delete**
  - one character from the remaining input
- **Insert**
  - Missing character into the remaining input
- **Replace**
  - Character by another character
- **Transpose**
  - Two adjacent characters

- **<u>Example</u> : *Divide the following C++ program:***

```
float limitedSquare(x) { float x;
  /* returns x-squared, nut never more than 100 */
  return (x <= -10.0 || x >= 10.0) ? 100 : x*x;
}
```

into appropriate lexemes, Which lexemes should get associated lexical values? What should those values be?

- **Solution:**

<float> <id, limitedSquare> <(> <id, x> <)> <{> <float> <id, x>

- <return> <(> <id, x> <op,"<="> <num, -10.0> <op, "||"> <id, x> <op, ">="> <num, 10.0> <)> <op, "?"> <num, 100> <op, ":"> <id, x> <op, "*"> <id, x>
- <}>

Input buffering

- Speed up the reading the source program.
- Sometimes lexical analyzer needs to look ahead some symbols to decide about the token to return
  - In C, single-character operators like - , =, or < could also be the beginning of a two-character operator like - > , ==, or <=.

https://www.slideshare.net/dattatraygandhmal/input-buffering

- **Buffer Pairs**
- Buffering techniques have been developed to reduce the amount of overhead required to process a single input character.
- We need to introduce a two buffer scheme to handle large look-aheads safely

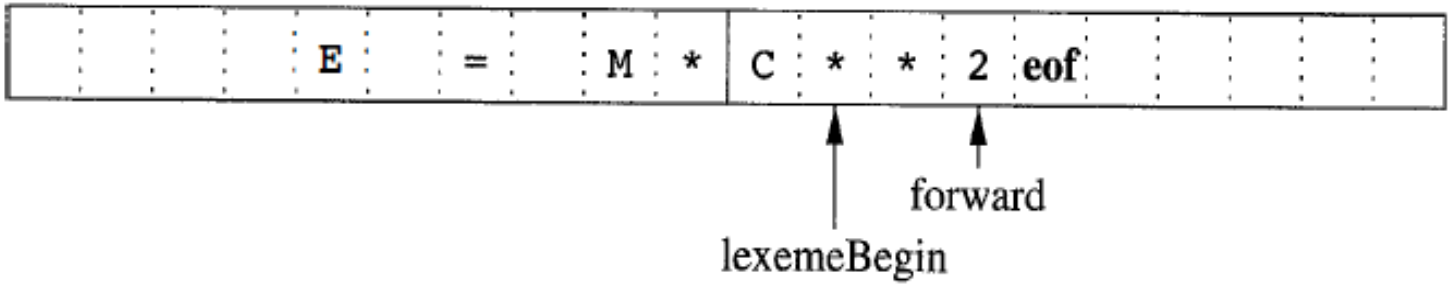
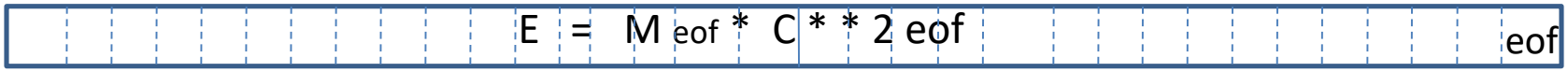


Figure : Using a pair of input buffers

Sentinels




```
Switch (*forward++) {
    case eof:
        if (forward is at end of first buffer) {
            reload second buffer;
            forward = beginning of second buffer;
        }
        else if {forward is at end of second buffer) {
            reload first buffer;\
            forward = beginning of first buffer;
        }
        else /* eof within a buffer marks the end of input */
            terminate lexical analysis;
        break;
    cases for the other characters;
}
```

Specification of tokens

- In theory of compilation, **regular expressions** are used to formalize the specification of tokens
- **Regular expressions** are means for specifying regular languages
- Example:
  - letter(letter | digit)*
- Each regular expression is a pattern specifying the form of strings

Strings and Languages

- String means a **finite sequence of symbols**.
- For example,

  computer (c, o, m, p, u, t, e, r)
- Symbols are given through alphabet. An alphabet is a finite set of symbols
- ***alphabet*** *or* ***character class*** : any finite set of symbols.
- e.g., set **{0,1}** → **binary alphabet**.
- ***sentence*** *and* ***word*** : used as synonyms for the term string.

Strings and Languages

- The ***length of a string s*** : **| s |** → number of occurrences of symbols in ***s***.
- e.g., string "**cs6660**" → length six.
- The ***empty string*** : **ε** – length of empty string is **zero.**
- ***language*** : any set of strings over some fixed alphabet.
- e.g., **{ε}** – set containing only empty string is language under **ϕ.**

Operations on string

1. ***Concatenation of string***

   If *x* and *y* are strings, then
   the *concatenation* of *x* and *y* (xy)→ the string formed by appending *y* to *x*.

x* = *hello and ***y* = *world***; then ***xy*** is **helloworld**.

2. **Exponentiation** Let s be a string, then

   $s^0 = \varepsilon$, $s^1 = s$, $s^2 = ss$, $s^3 = sss$, …

   $s^n = sss$…**s(n times)**

3. **Identity element**

$s\varepsilon = \varepsilon s = s$.

Operations on Strings

TERM	DEFINITION
Prefix of s	A string obtained by removing zero or more trailing symbols of string s; **e.g., cs is a prefix of cs6660.**
Suffix of s	A string formed by deleting zero or more of the leading symbols of s; **e.g., 660 is a suffix of cs6660.**
Substring of s	A string obtained by deleting a prefix and a suffix from s; **e.1`g., 66 is a substring of cs6660.**
Proper prefix, suffix, or substring of s	Any nonempty string x that is a prefix, suffix or substring of s that **s <> x.**
Subsequence of s	Any string formed by deleting zero or more not necessarily contiguous symbols from s; **e.g., c60 is a subsequence of cs6660**.

Operations on Languages

OPERATION	**DEFINITION**
Union of L and M. written LυM	L υ M = { s | s is in L or s is in M }
Concatenation of L and M. written LM	LM = { st | s is in L and t is in M }
Kleene closure of L. written L*	L* denotes "zero or more concatenation of" L. $L^* = \bigcup_{i=0}^{\infty} L^i$
Positive closure of L. written L+	L^+ denotes "one or more Concatenation of" L. $L^+ = \bigcup_{i=1}^{\infty} L^i$

Regular expressions

- ε is a regular expression, $L(\varepsilon) = \{\varepsilon\}$
- If a is a symbol in Σ, then a is a regular expression, $L(a) = \{a\}$
- (r) | (s) is a regular expression denoting the language $L(r) \cup L(s)$
- (r)(s) is a regular expression denoting the language $L(r)L(s)$
- (r)* is a regular expression denoting $(L(r))^*$
- (r) is a regular expression denting L(r)

Regular definitions

- For notational convenience, names are given to RE and use their names in subsequent expressions

d1 -> r1

d2 -> r2

…

dn -> rn

- Example: regular definition for the language of C identifiers

letter_ -> A | B | … | Z | a | b | … | Z |_

digit -> 0 | 1 | … | 9

id -> letter_ (letter_| digit)*

Extensions

- One or more instances: $(r)^+$
- Zero of one instances: r?
- Character classes: [abc]

- Example:
  - letter_ -> [A-Za-z_]
  - digit -> [0-9]
  - id -> letter_(letter_|digit)*

Example Unsigned numbers (integer or floating point) are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.

- ***digit → 0 | 1 | • • • | 9***
- ***digits → digit digit****
- ***optionalFraction → . digits |*** ε
- ***optionalExponent → (E (+ | - |*** ε ***) digits) |*** ε
- ***number → digits optionalFraction optionalExponent***
- Using shorthands:
- **digit → [0-9]**
- ***digits*** **→ digit+**
- ***optionalFraction*** **→ (. digits)?**
- ***optionalExponent*** **→ (E [+-]? digits) ?**
- ***number*** **→** ***digits*** **(. digits)? (E [+ -]? digits) ?**

Recognition of tokens

- **Grammar for branching statements**

stmt -> **if** expr **then** stmt

| **if** expr **then** stmt **else** stmt

| ε

expr -> term **relop** term

| term

term -> **id**

| **number**

Recognition of tokens (cont.)

Grammar for branching statements

```
stmt -> if expr then stmt
      | if expr then stmt else stmt
      | ε
expr -> term relop term
      | term
term -> id
      | number
```

digit -> [0-9]

digits -> digit+

number -> digits(.digits)? (E[+-]? digits)?

letter -> [A-Za-z]

id -> letter (letter|digit)*

if -> if

then -> then

else -> else

relop -> < | > | <= | >= | = | <>

- Whitespaces:

  ws -> (blank | tab | newline)+

Attribute Values

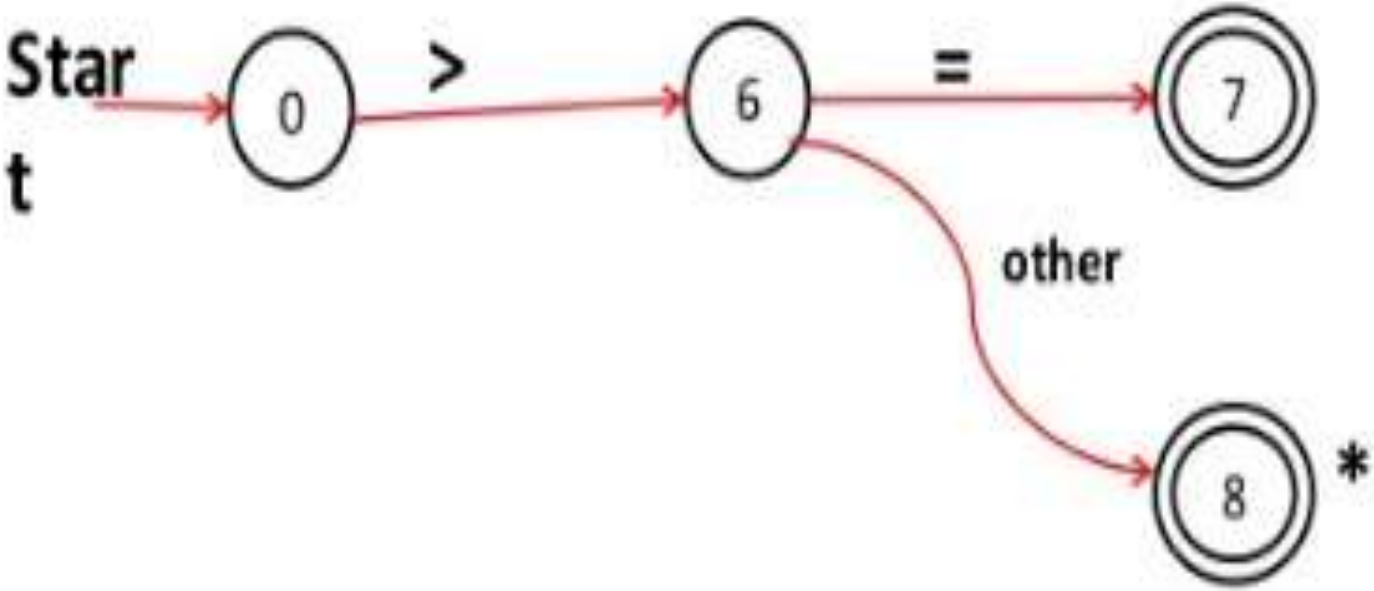
Lexeme	Token Name	Attribute Value
Any WS	-	-
if	If	-
then	Then	-
else	Else	-
Anyid	Id	pointer to table entry
Any number	Number	pointer to table entry
<	Relop	LT
<=	Relop	LE
==	Relop	EQ
<>	Relop	NE

Transition Diagrams

- Transition diagram : Intermediate step in construction of LA is to convert patterns into flowcharts.
- TD are also called finite automata
- We have a collection of STATES drawn as node in a graph.
- TRANSITIONS between states are represented by directed edges in the graph.
- Each transition leaving a state s is labeled with a set of input characters that can occur after state s.

Transition Diagrams (Cont..)

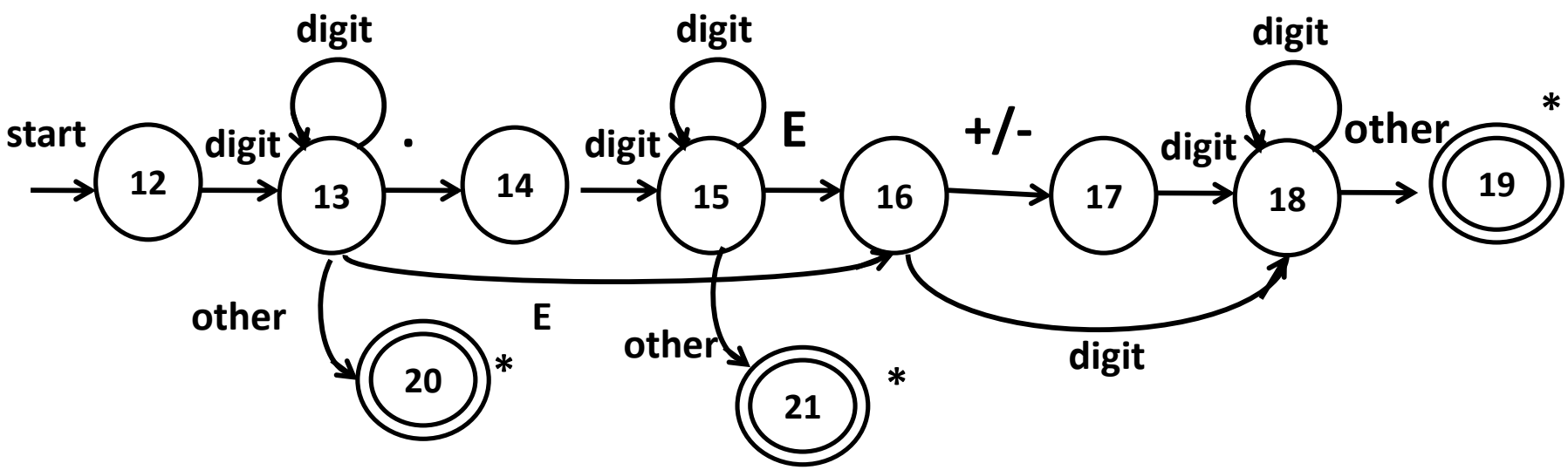
- For now, the transitions must be DETERMINISTIC.
- Each transition diagram has a single START state and a set of TERMINAL STATES.
- The label OTHER on an edge indicates all possible inputs not handled by the other transitions.
- Usually, when we recognize OTHER, we need to put it back in the source stream since it is part of the next token.
- This action is denoted with a * next to the corresponding state.


Star
t
0
>
6
=
7
other
8
*

Example: Unsigned numbers (integer or floating point) are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.

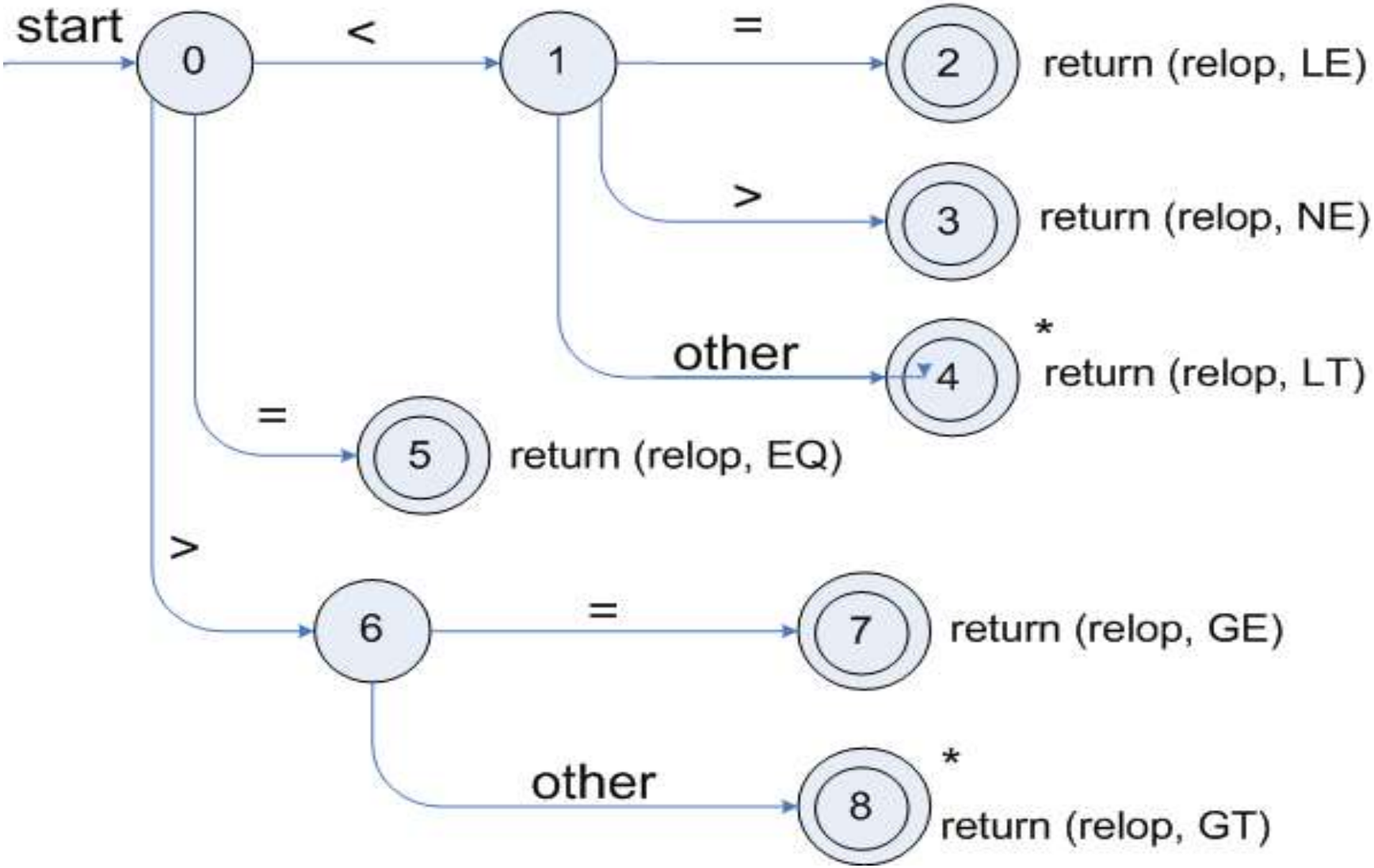
Transition Diagram: digits->digit$^+$

***number* → *digits* (. digits)? (E [+ -]? digits) ?**

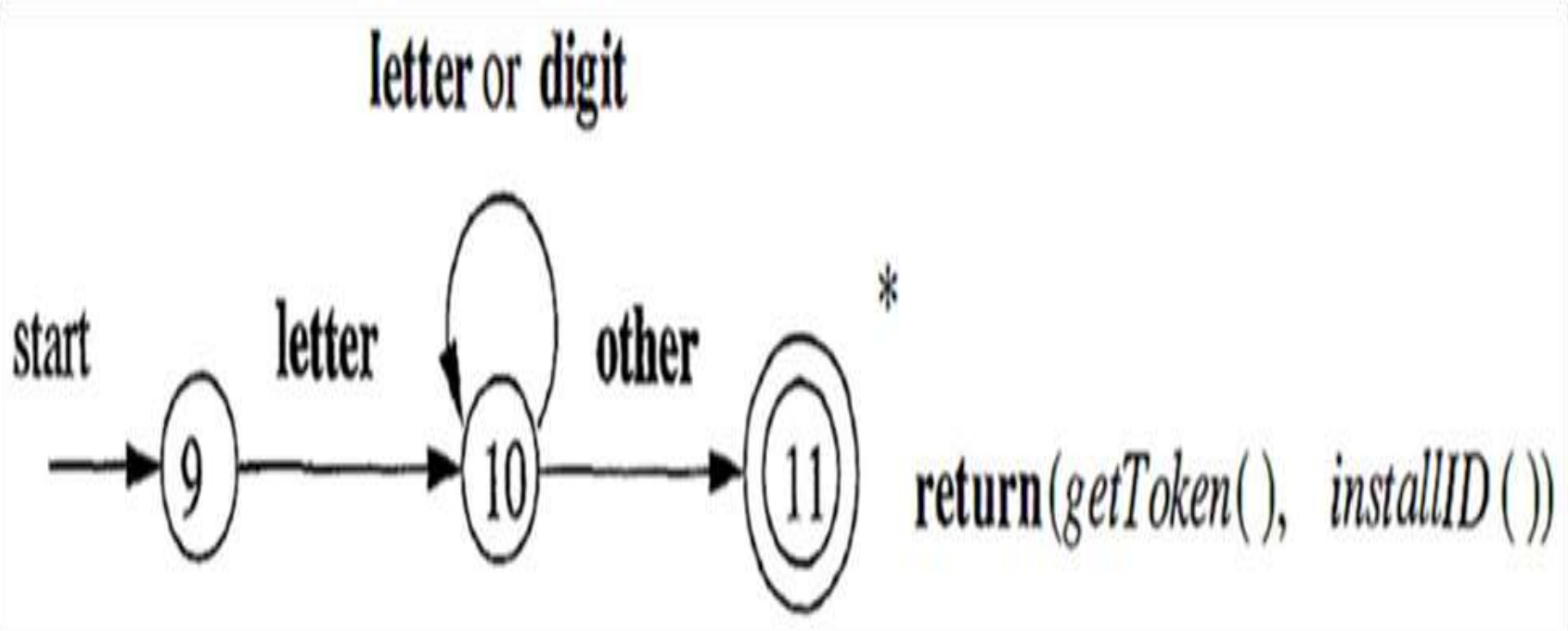


Transition Diagram for Relational operator :

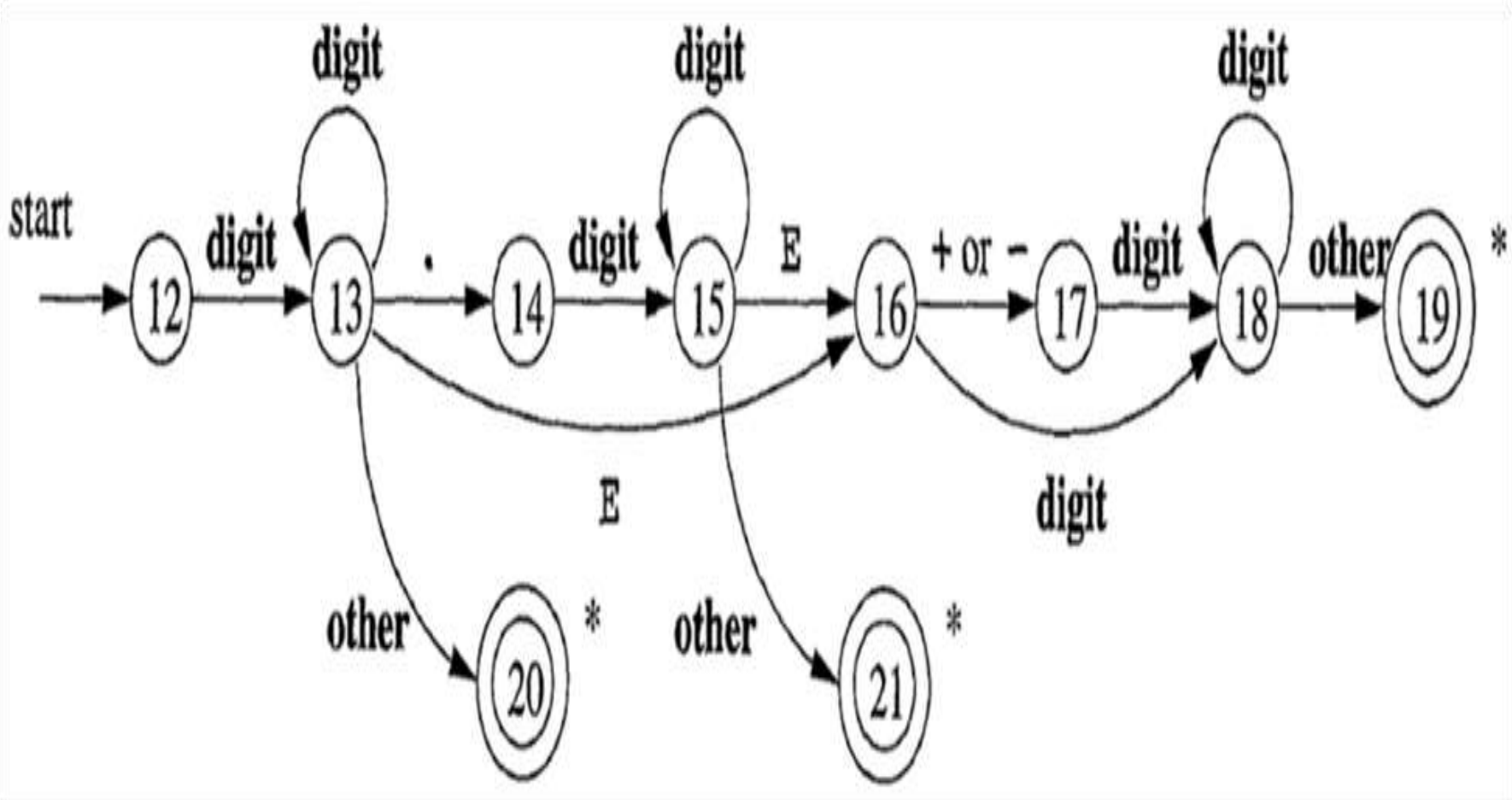
"< | > |< = | >= | = | <>"



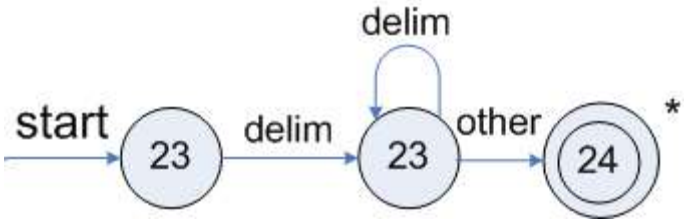
Transition diagrams for identifier



Transition Diagram for unsigned number



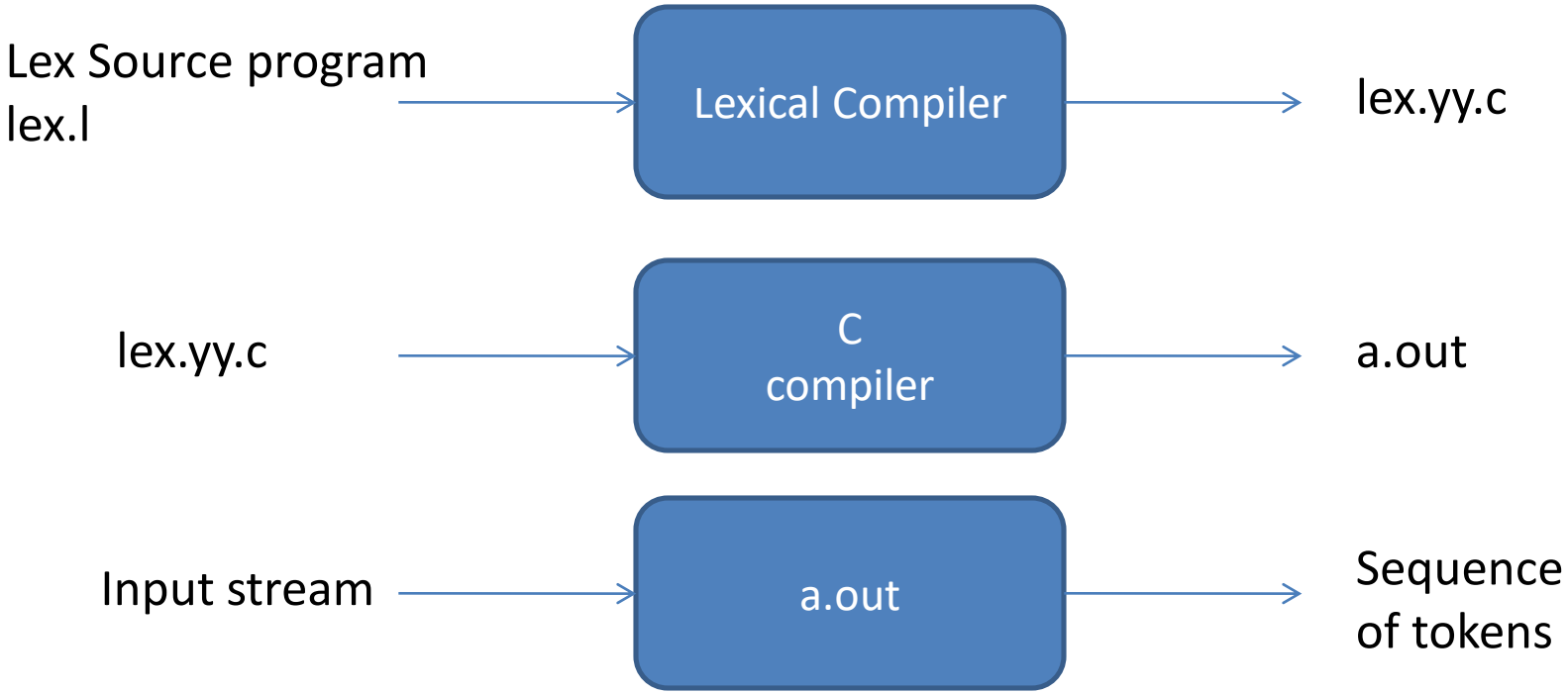
Transition diagram for whitespace



Architecture of a transition-diagram-based lexical analyzer

```
TOKEN getRelop()
{
    TOKEN retToken = new (RELOP)
    while (1) {             /* repeat character processing until a
                                    return or failure occurs    */
    switch(state) {
        case 0: c= nextchar();
                    if (c == '<') state = 1;
                    else if (c == '=') state = 5;
                    else if (c == '>') state = 6;
                    else fail();   /* lexeme is not a relop */
                    break;
        case 1: …
        …
        case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
    }
```

Lexical Analyzer Generator - Lex



Structure of Lex programs

declarations
%%
translation rules
%%
auxiliary functions

→ Pattern {Action}

Example

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}

/* regular definitions
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
{ws} {/* no action and no return */}
if          {return(IF);}
then{return(THEN);}
else {return(ELSE);}
{id}  {yylval = (int) installID(); return(ID); }
{number}    {yylval = (int) installNum(); return(NUMBER);}
…
```

```
Int installID() {/* funtion to install the
    lexeme, whose first character is
    pointed to by yytext, and whose
    length is yyleng, into the symbol
    table and return a pointer thereto
    */
}

Int installNum() { /* similar to
    installID, but puts numerical
    constants into a separate table */
}
```

Finite Automata

- Regular expressions = specification
- Finite automata = implementation

- A finite automaton consists of
  - An input alphabet Σ
  - A set of states S
  - A start state n
  - A set of accepting states $F \subseteq S$
  - A set of transitions state $\rightarrow^{\text{input}}$ state

Finite Automata

- Transition

$$s_1 \rightarrow^a s_2$$

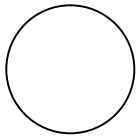
- Is read

In state s_1 on input "a" go to state s_2

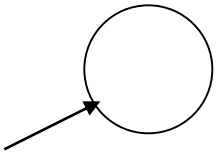
- If end of input
  - If in accepting state => accept, othewise => reject
- If no transition possible => reject

Finite Automata State Graphs

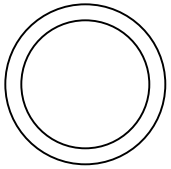
- A state



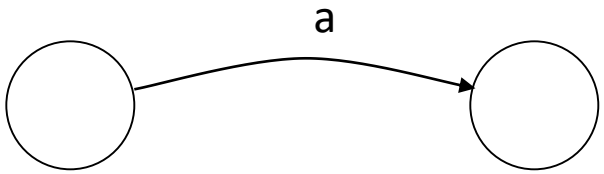
- The start state



- An accepting state



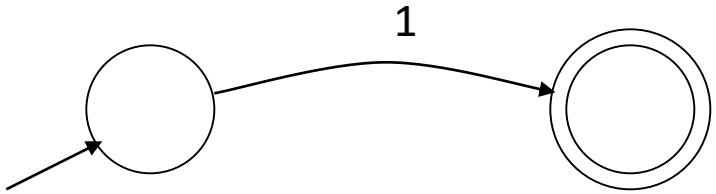
- A transition



A Simple Example

- A finite automaton that accepts only “1”

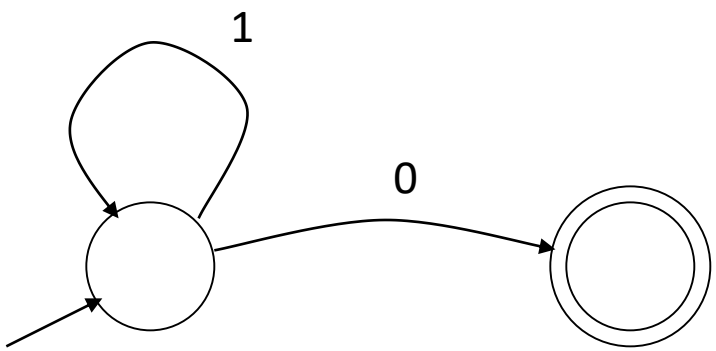
- A finite automaton accepts a string if we can follow transitions labeled with the characters in the string from the start to some accepting state



Another Simple Example

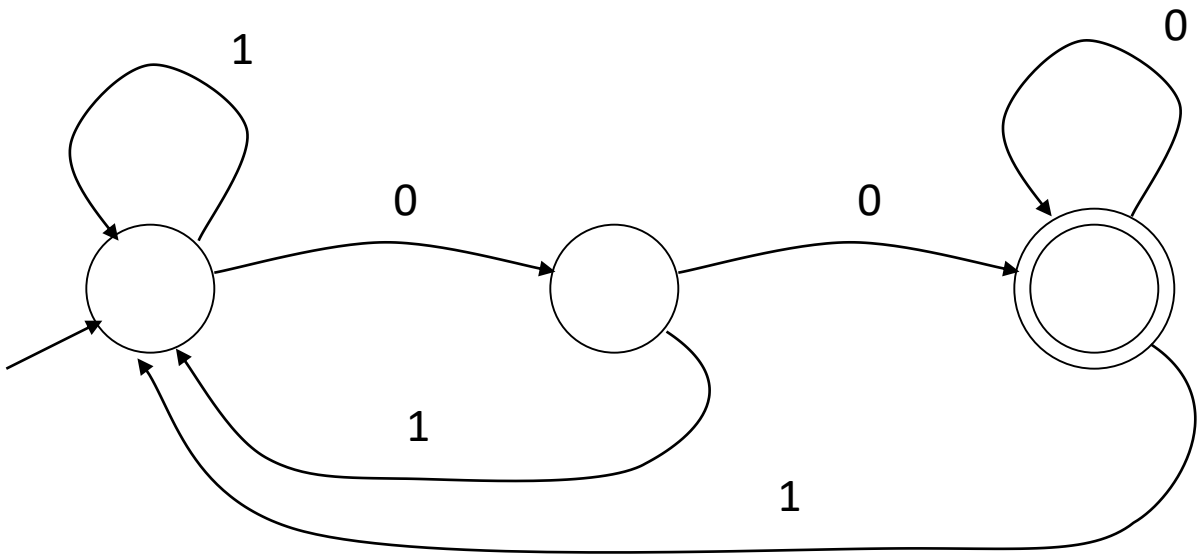
- A finite automaton accepting any number of 1's followed by a single 0
- Alphabet: {0,1}

- Check that "1110" is accepted but "101…" is not



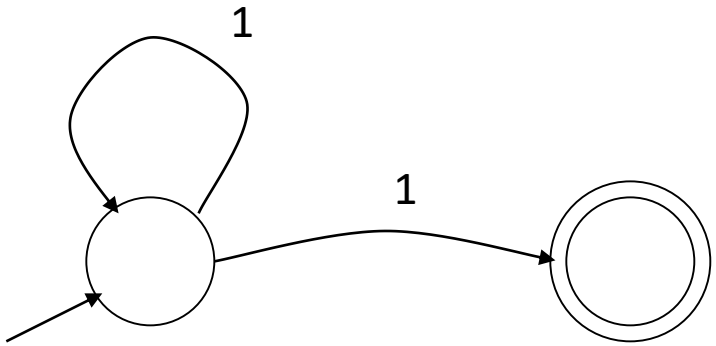
And Another Example

- Alphabet {0,1}
- What language does this recognize?



And Another Example

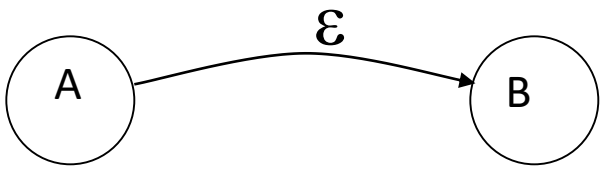
- Alphabet still { 0, 1 }



- The operation of the automaton is not completely defined by the input
  - On input “11” the automaton could be in either state

Epsilon Moves

- Another kind of transition: ε-moves



- Machine can move from state A to state B without reading input

Deterministic and Nondeterministic Automata

- Deterministic Finite Automata (DFA)
  - One transition per input per state
  - No ε-moves
- Nondeterministic Finite Automata (NFA)
  - Can have multiple transitions for one input in a given state
  - Can have ε-moves
- *Finite* automata have *finite* memory
  - Need only to encode the current state

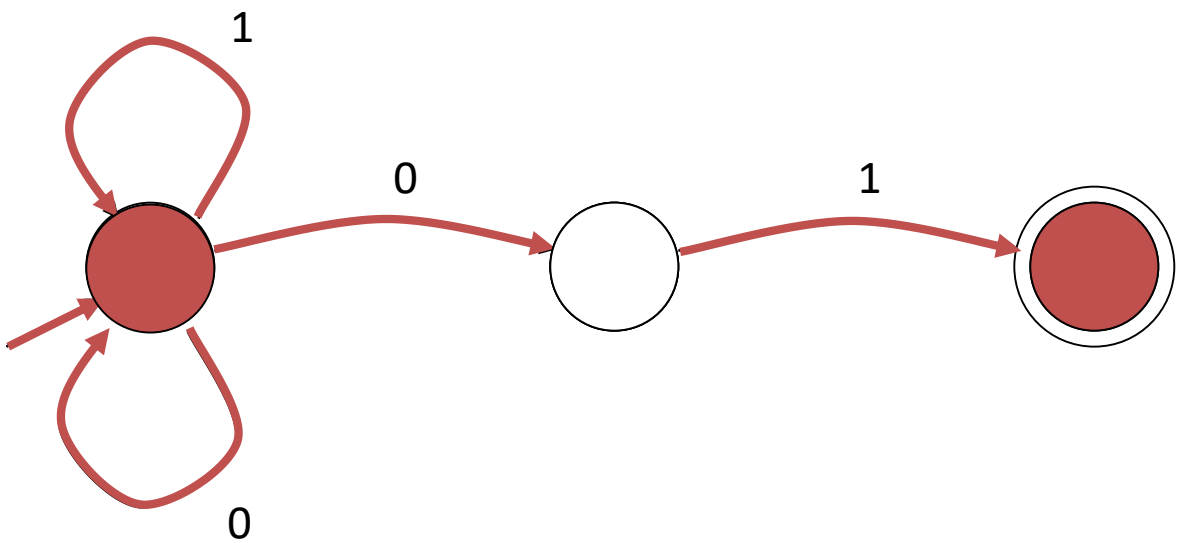
Execution of Finite Automata

- A DFA can take only one path through the state graph
  - Completely determined by input

- NFAs can choose
  - Whether to make ε-moves
  - Which of multiple transitions for a single input to take

Acceptance of NFAs

- An NFA can get into multiple states



- Input: 1 0 1
- Rule: NFA accepts if it <u>can</u> get in a final state

NFA vs. DFA (1)

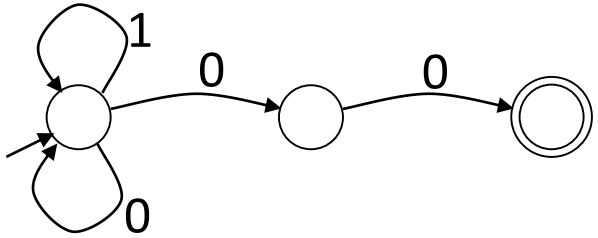
- NFAs and DFAs recognize the same set of languages (regular languages)

- DFAs are easier to implement
  - There are no choices to consider

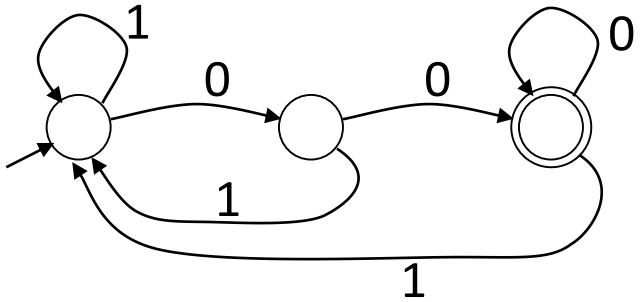
NFA vs. DFA (2)

- For a given language the NFA can be simpler than the DFA

NFA



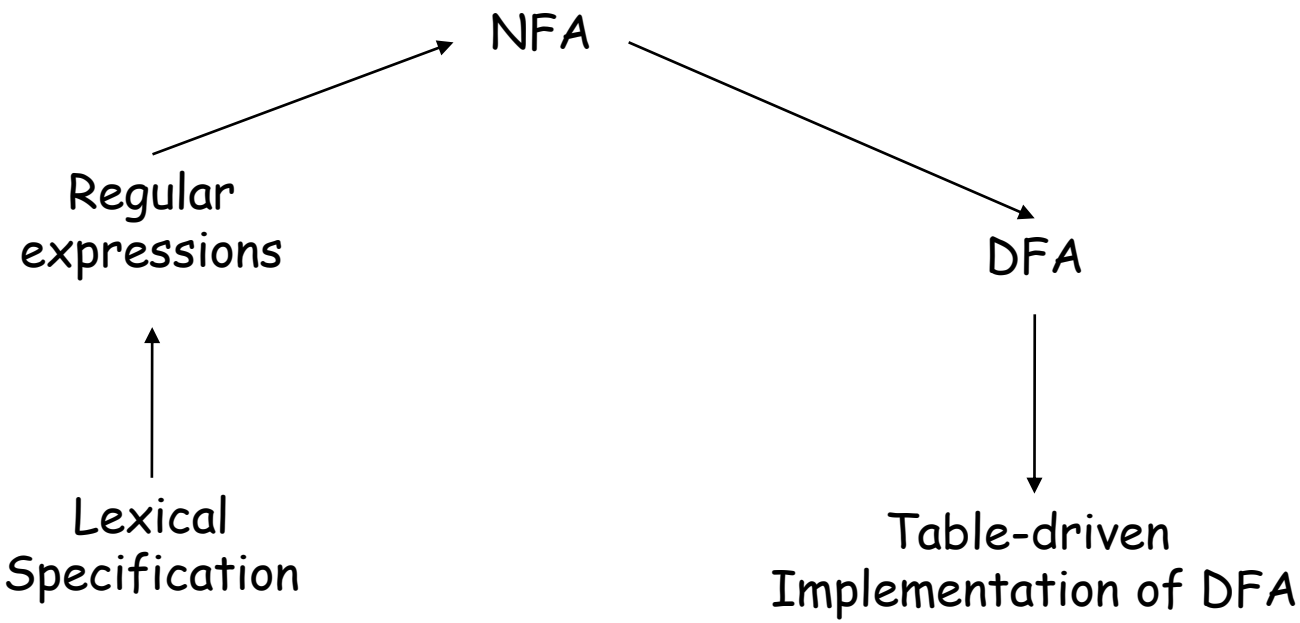
DFA



- DFA can be exponentially larger than NFA

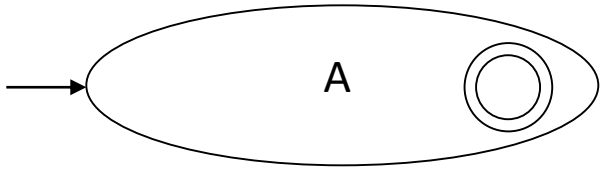
Regular Expressions to Finite Automata

- High-level sketch

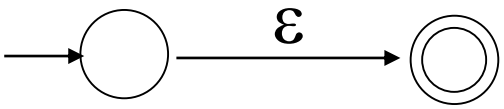


Regular Expressions to NFA (1)

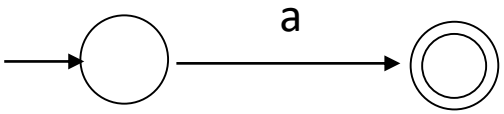
- For each kind of rexp, define an NFA
  - Notation: NFA for rexp A



- For ε

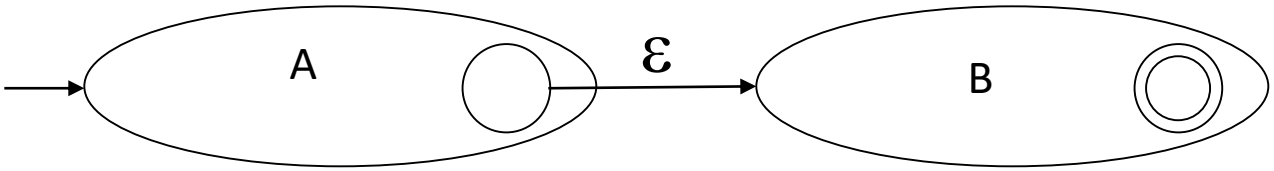


- For input a

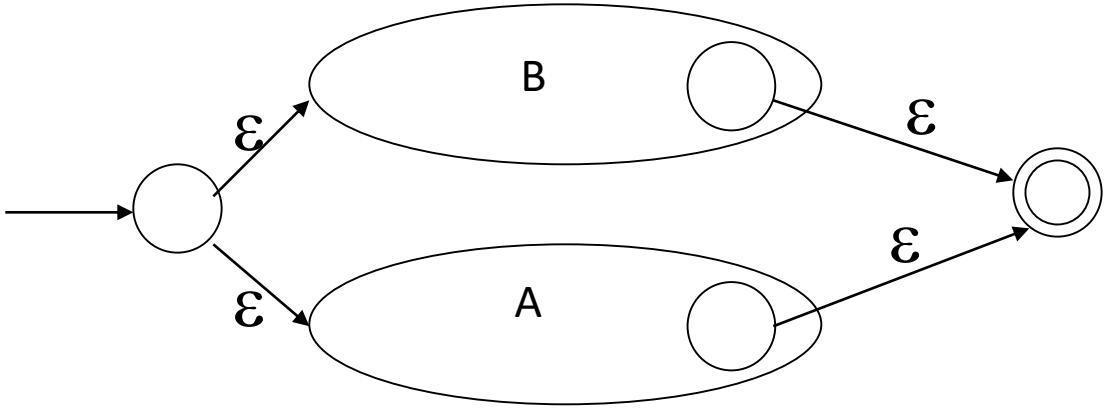


Regular Expressions to NFA (2)

- For AB

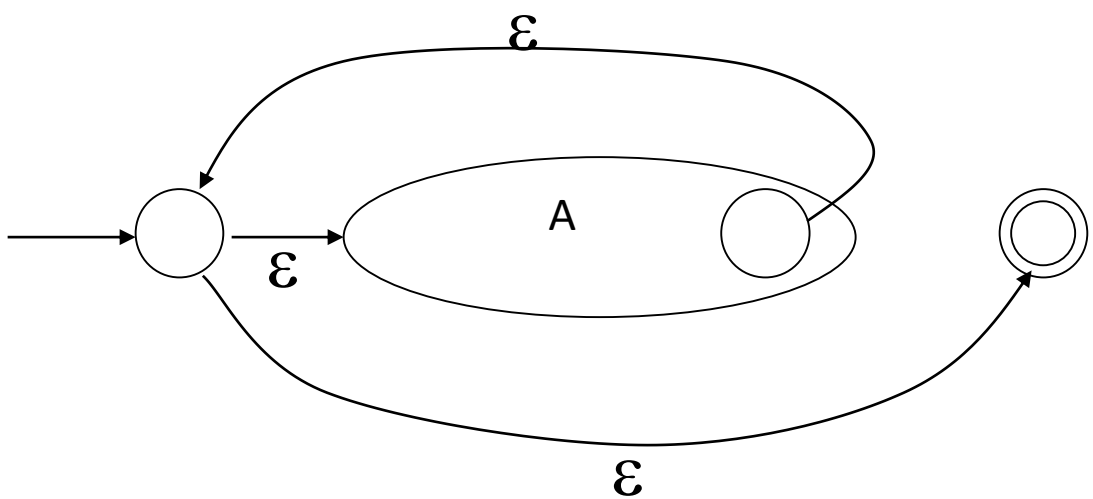


- For A | B



Regular Expressions to NFA (3)

- For A*

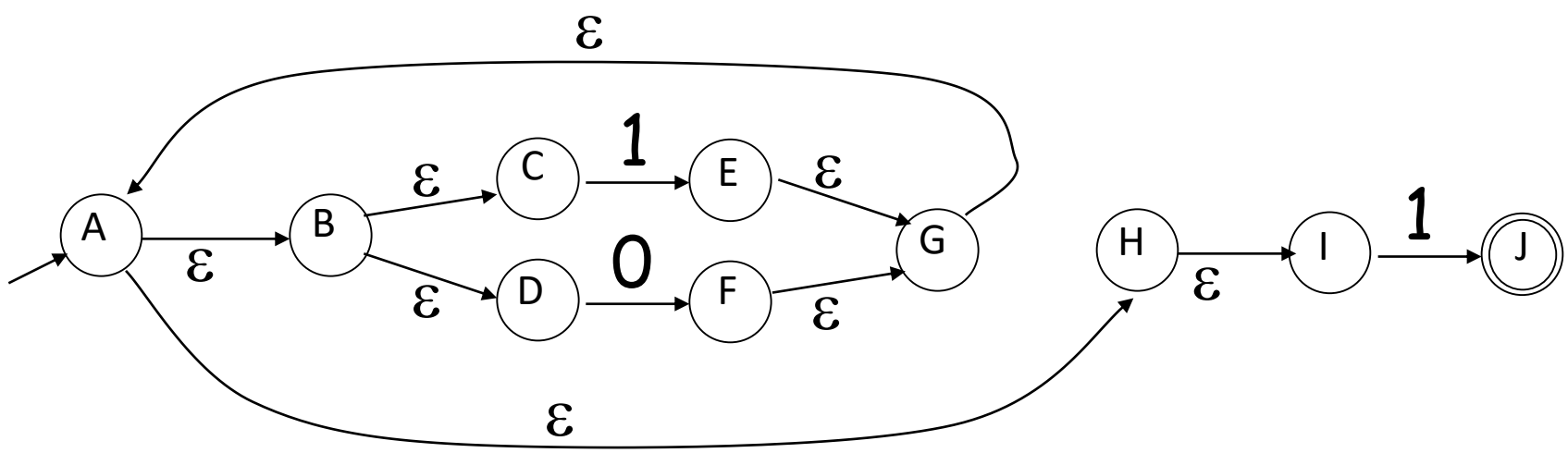


Example of RegExp -> NFA conversion

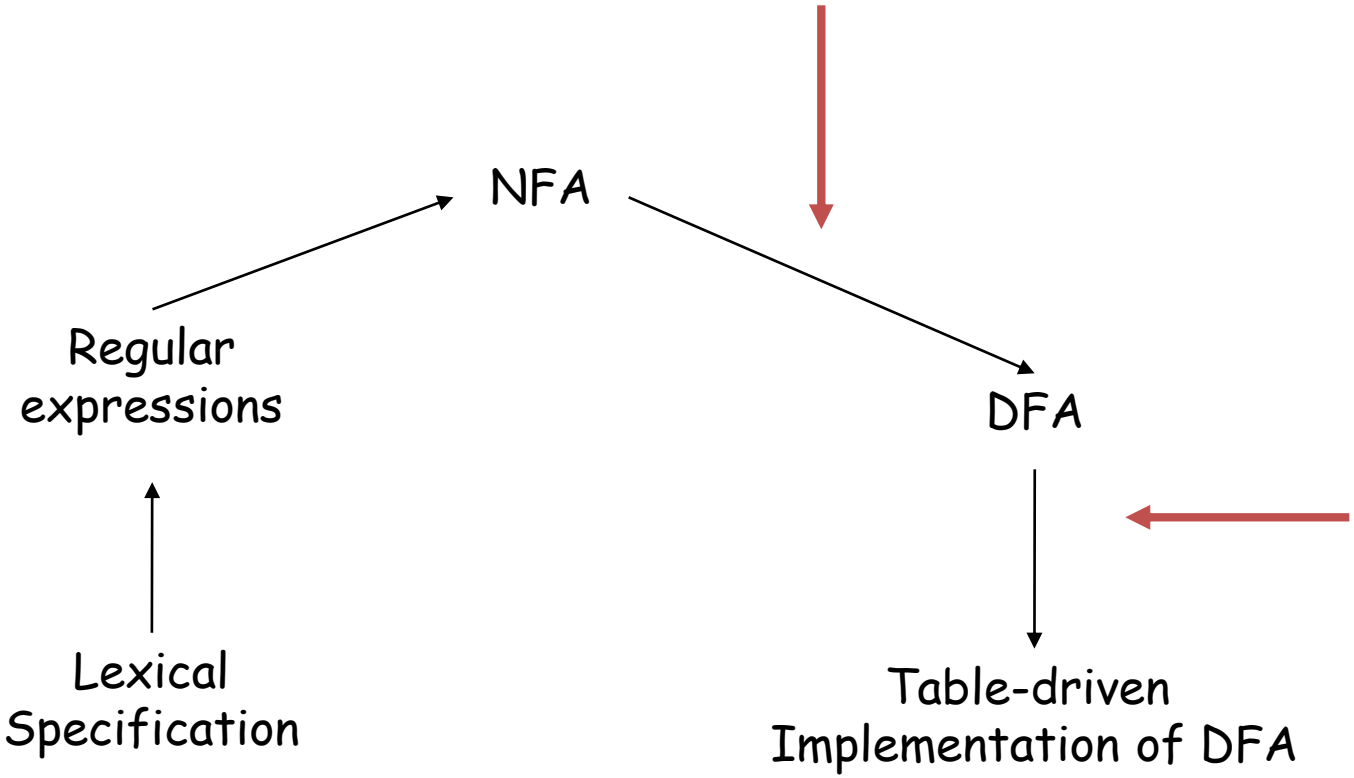
- Consider the regular expression

(1 | 0)*1

- The NFA is



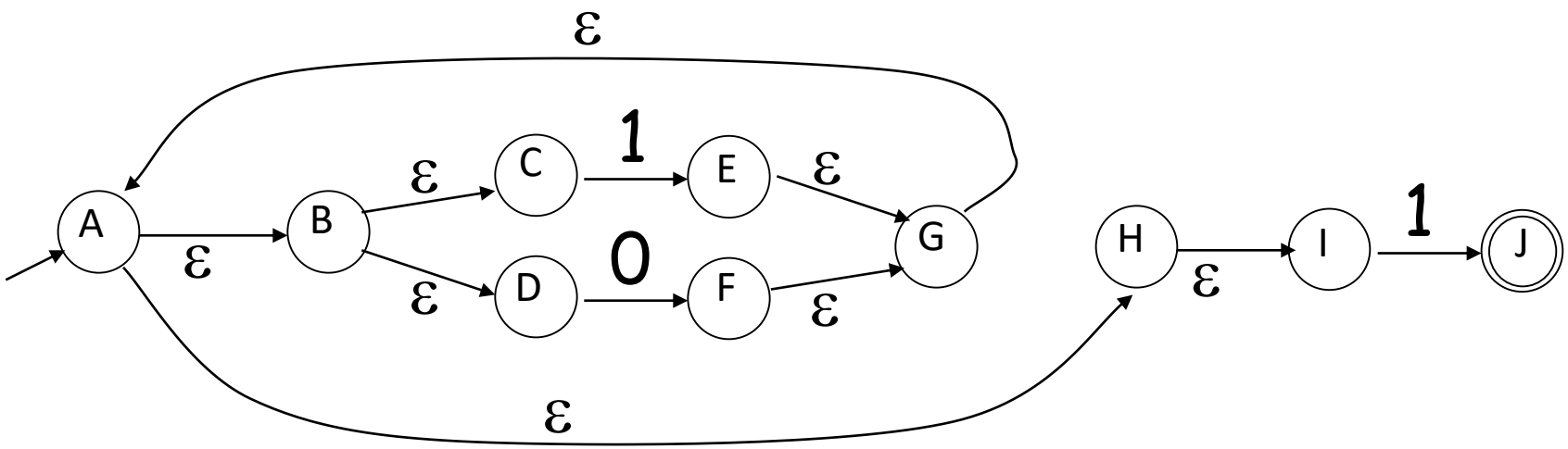
Next

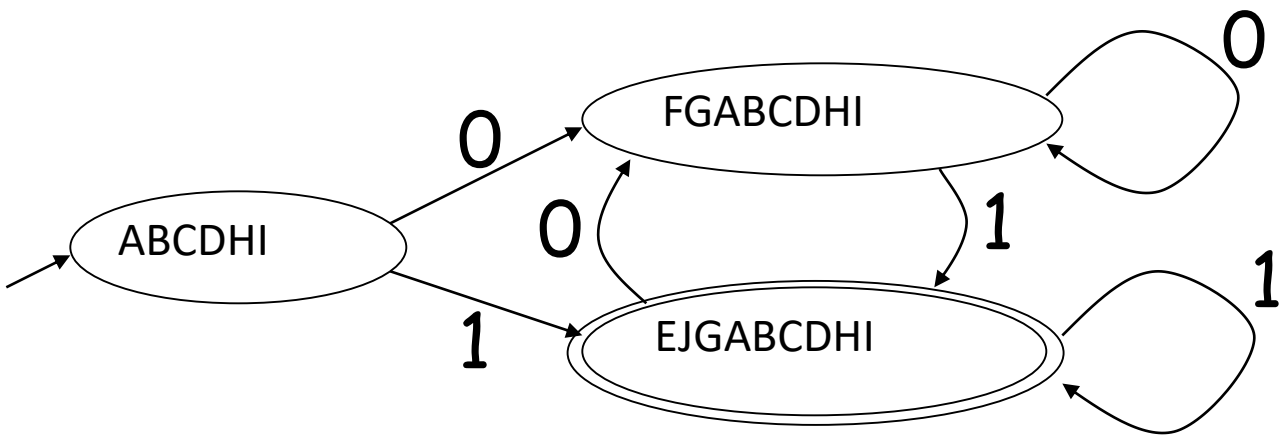

NFA
Regular expressions
DFA
Lexical Specification
Table-driven Implementation of DFA

NFA to DFA. The Trick

- Simulate the NFA
- Each state of resulting DFA
  = a non-empty subset of states of the NFA
- Start state
  = the set of NFA states reachable through ε-moves from NFA start state
- Add a transition $S \rightarrow^{a} S'$ to DFA iff
  – S' is the set of NFA states reachable from the states in S after seeing the input a
    - considering ε-moves as well

NFA -> DFA Example





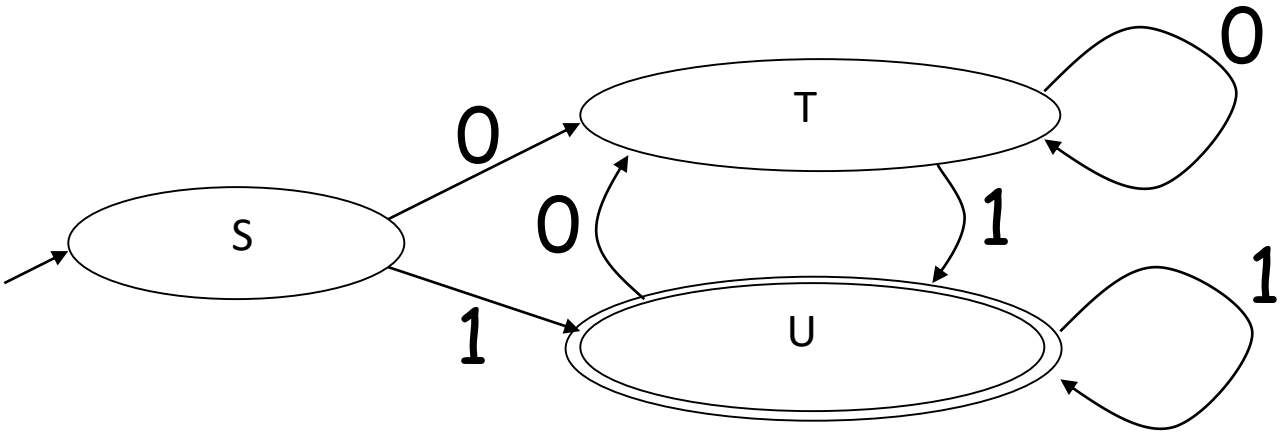
NFA to DFA. Remark

- An NFA may be in many states at any time
- How many different states ?
- If there are N states, the NFA must be in some subset of those N states
- How many non-empty subsets are there?
  - $2^N - 1$ = finitely many, but exponentially many

Implementation

- A DFA can be implemented by a 2D table T
  - One dimension is "states"
  - Other dimension is "input symbols"
  - For every transition $S_i \rightarrow^a S_k$ define T[i,a] = k
- DFA "execution"
  - If in state S_i and input a, read T[i,a] = k and skip to state S_k
  - Very efficient

Table Implementation of a DFA



	0	1
S	T	U
T	T	U
U	T	U

Implementation (Cont.)

- NFA -> DFA conversion is at the heart of tools such as flex or jflex
- But, DFAs can be huge
- In practice, flex-like tools trade off speed for space in the choice of NFA and DFA representations

OPTIMIZATION OF DFA-BASED PATTERN MATCHERS

(CONVERTING A REGULAR EXPRESSION DIRECTLY TO A DFA)

- **Algorithm: Convert Regular Expression Directly To a DFA**
- Input : a regular expression r
- Output : A DFA D that recognizes L(r)
- **Method**
- Construct the syntax tree of (r) #
- Compute nullable, firstpos, lastpos, followpos
- Put firstpos(root) into the states of DFA as an unmarked state.
- *while* (there is an unmarked state ***S*** in the states of DFA) *do*
  - mark ***S***
  - *for each* input symbol ***a*** *do*
    - let $s_1,\ldots,s_n$ are positions in ***S*** and symbols in those positions are ***a***
    - ***S'*** ß followpos(s_1) È … È followpos(s_n)
    - **Dtran**[***S,a***] ß ***S'***
    - if (***S'*** is not in the states of DFA)
      - – put ***S'*** into the states of DFA as an unmarked state.
- *the* ***start state*** *of DFA is* ***firstpos(root)***
- *the* ***accepting states*** *of DFA are all states containing the* ***position of #***

- **Functions computed from the syntax tree**
- In order to construct a DFA directly from the regular expression we have to:
  - Build the syntax tree
  - Compute functions for finding the positions
    - Firstpos, Lastpos, Followpos.
- Find D_{tran}
- Optimized DFA

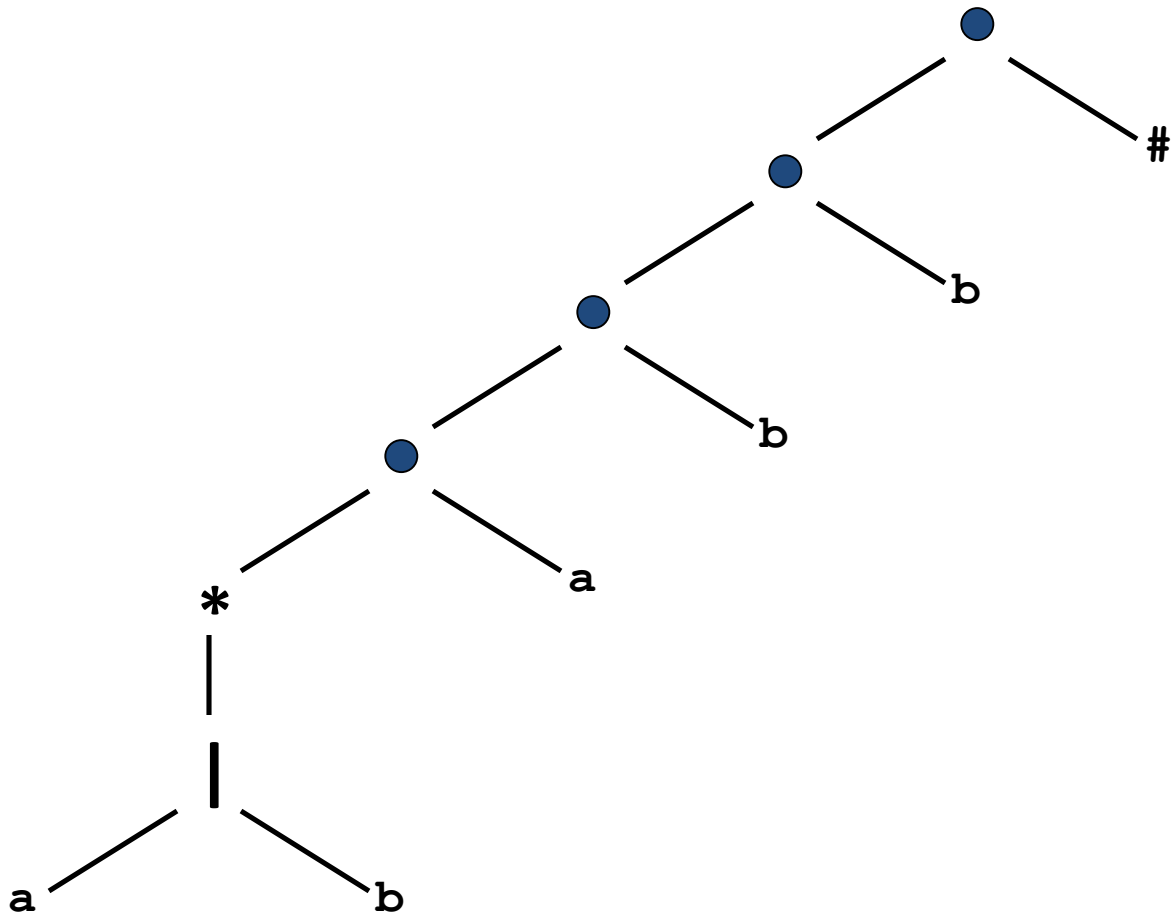
Compute four functions referring (r)#

- **nullable(n)**
  - true for syntax tree node n if the sub expression represented by n
    - has ε in its language
    - can be made null or the empty string even it can represent other strings
- **firstpos(n)**
  - set of positions in the n rooted subtree that correspond to the first symbol of at least one string in the language of the subexpression rooted at n
- **lastpos(n)**
  - set of positions in the n rooted subtree that correspond to the last symbol of at least one string in the language of the subexpression rooted at n
- **followpos(n)**
  - for a position p
  - is the set of positions q such that
  - $x=a_1a_2\ldots a_n$ in L((r)#) such that
  - for some i there is a way to explain the membership of x in L((r)#) by matching a_i to position p of the syntax tree a_{i+1} to position q

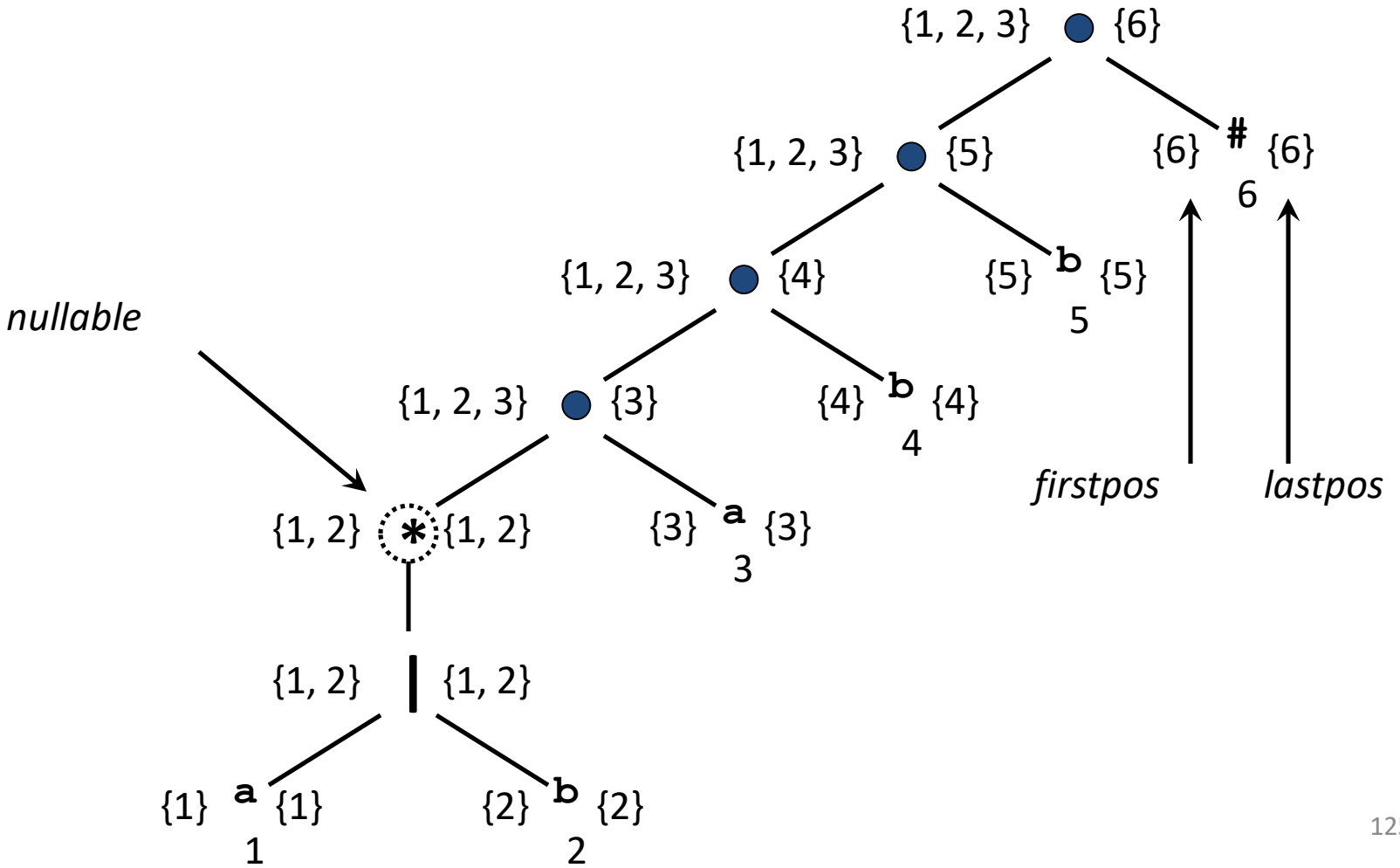
From Regular Expression to DFA Directly: Annotating the Tree

Node n	*nullable*(n)	*firstpos*(n)	*lastpos*(n)
Leaf ε	true	$\varnothing$	$\varnothing$
Leaf i	false	$\{i\}$	$\{i\}$
| / \\ c_1 c_2	*nullable*(c_1) or *nullable*(c_2)	*firstpos*(c_1) $\cup$ *firstpos*(c_2)	*lastpos*(c_1) $\cup$ *lastpos*(c_2)
• / \\ c_1 c_2	*nullable*(c_1) and *nullable*(c_2)	**if** *nullable*(c_1) **then** *firstpos*(c_1) $\cup$ *firstpos*(c_2) **else** *firstpos*(c_1)	**if** *nullable*(c_2) **then** *lastpos*(c_1) $\cup$ *lastpos*(c_2) **else** *lastpos*(c_2)
* | c_1	true	*firstpos*(c_1)	*lastpos*(c_1)

From Regular Expression to DFA Directly: 1.Syntax Tree of (a|b)*abb#



From Regular Expression to DFA Directly: Syntax Tree of `(a|b)*abb#`



From Regular Expression to DFA Directly: *followpos*

Computing Followpos

A position of a regular expression can follow another position in two ways:

if n is a cat-node c_1c_2 (rule 1)

for every position i in lastpos(c_1) all positions in firstpos(c_2) are in followpos(i)

if n is a star-node (rule 2)

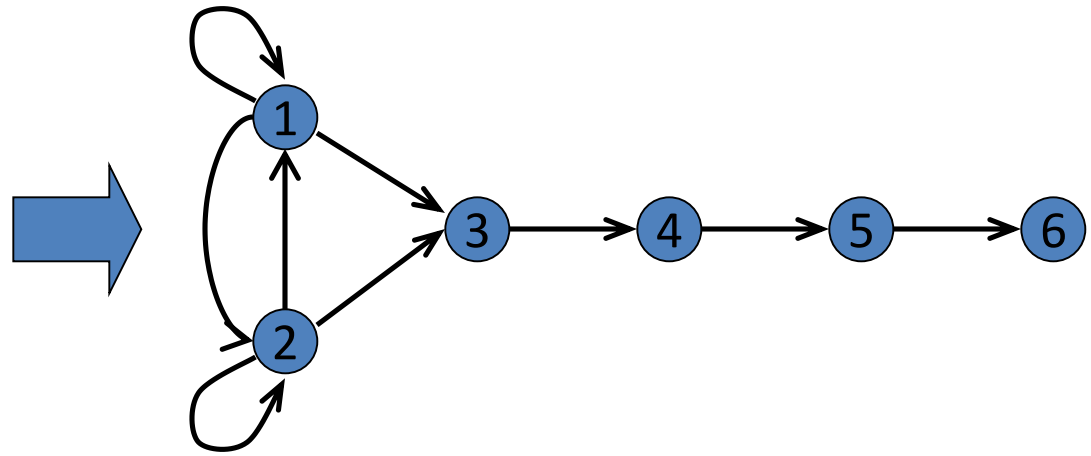
if i is a position in lastpos(n) then all positions in firstpos(n) are in followpos(i)

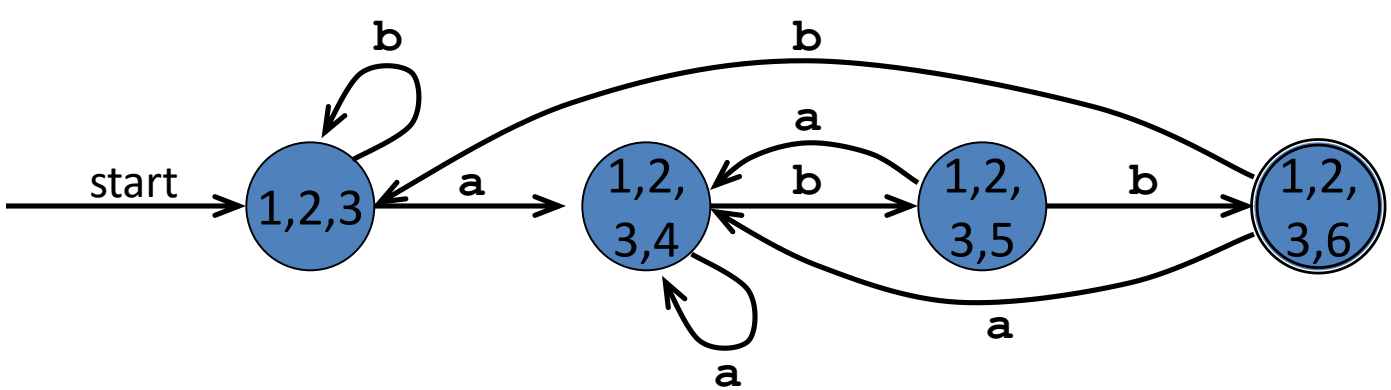
From Regular Expression to DFA Directly: Algorithm

s_0 := *firstpos*(*root*) where *root* is the root of the syntax tree
Dstates := {s_0} and is unmarked
while there is an unmarked state *T* in *Dstates* **do**
    mark *T*
    for each input symbol $a \in \Sigma$ **do**
        let *U* be the set of positions that are in *followpos*(*p*)
            for some position *p* in *T*,
            such that the symbol at position *p* is *a*
        if *U* is not empty and not in *Dstates* **then**
            add *U* as an unmarked state to *Dstates*
        end if
        Dtran[*T,a*] := *U*
    end do
end do

From Regular Expression to DFA Directly: Example

Node	*followpos*
1	{1, 2, 3}
2	{1, 2, 3}
3	{4}
4	{5}
5	{6}
6	-



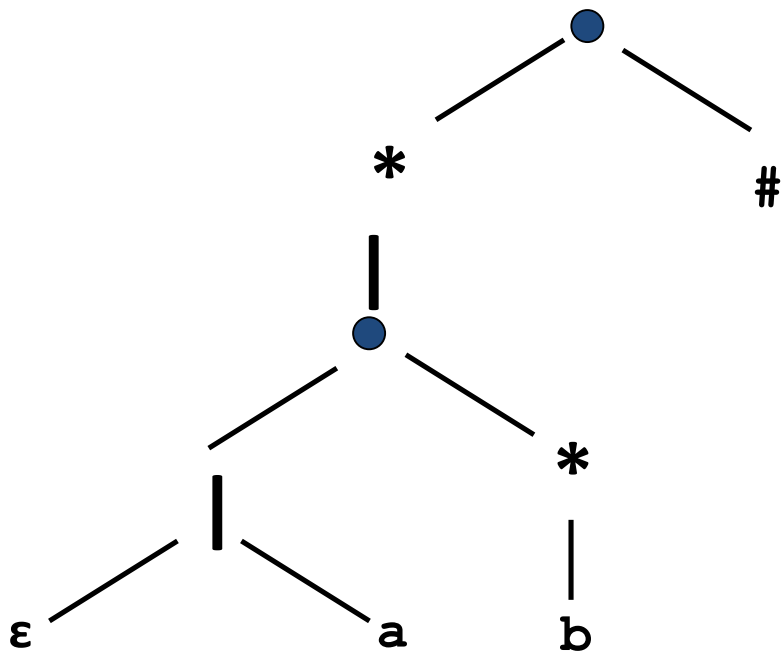


Regular Expression to DFA Directly: $((\varepsilon|a)b^*)^*$

Step-1 : Augmented Regular Expression:

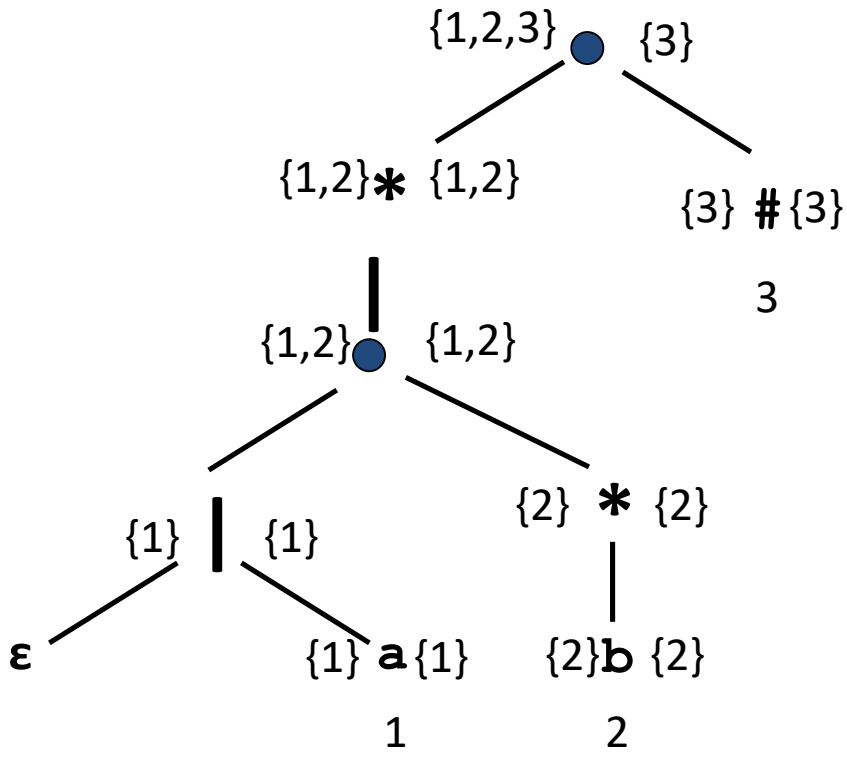
$((\varepsilon|a)b^*)^*\#$

Step-2:Syntax Tree of $((\varepsilon|a)b^*)^*\#$



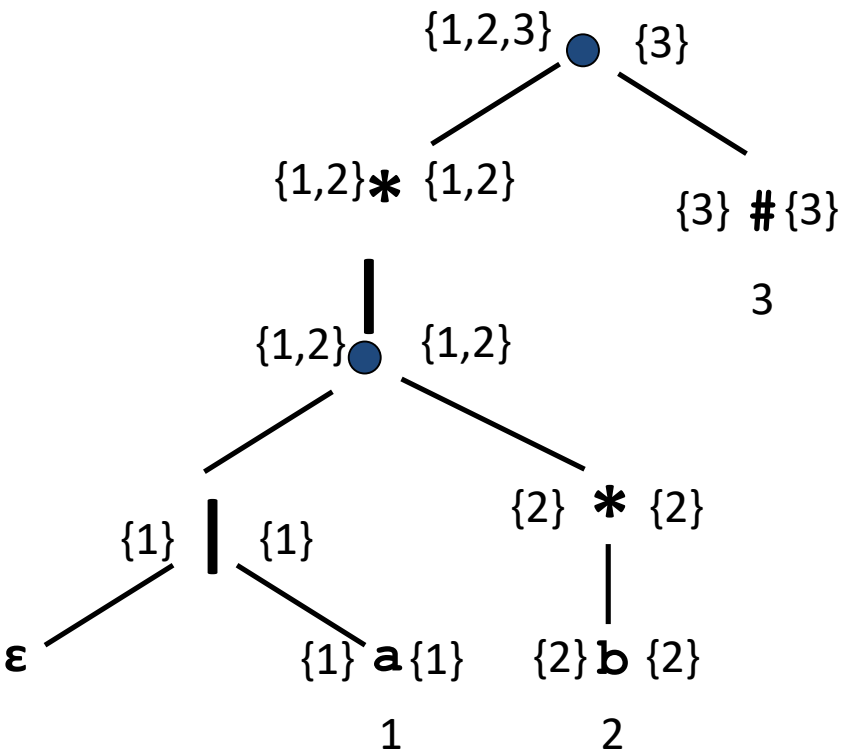
Regular Expression to DFA Directly: $((\varepsilon|a)b^*)^*$

Step 3: Compute Firstpos and Lastpos



Regular Expression to DFA Directly: $((\varepsilon|a)b^*)^*$

Step 4: Compute Followpos



Position(Node)	Followpos
1	{ 1,2,3}
2	{1,2,3 }
3	-

Find Positions for a & b

a position = 1

b position = 2

Step 5:- Find D_{tran}

Firstpos(n_o)={1,2,3}

=.....A

Dtran[A,a]= followpos(1)

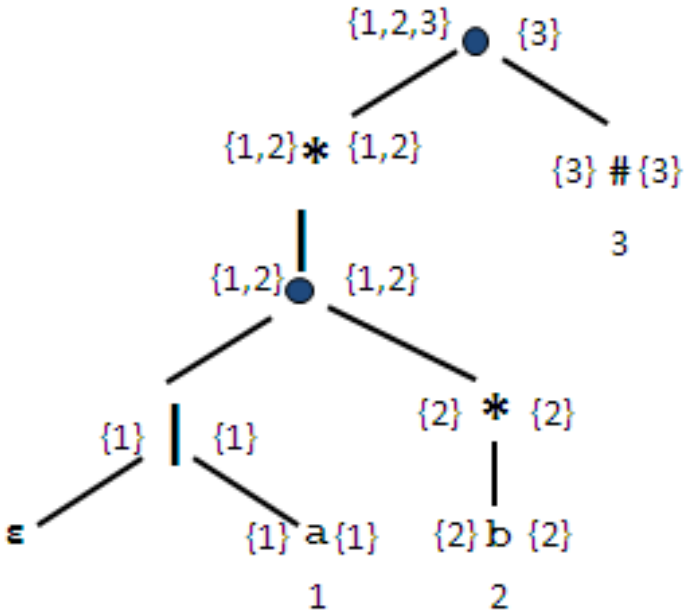
= {1,2,3}

=.....A

Dtran[A,b]= followpos(2)

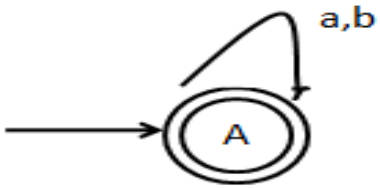
={1,2,3}

=.....A



Position(Node)	Followpos
1	{ 1,2,3}
2	{1,2,3}
3	-

Step 6:- Optimized DFA Transition Table

States/Input	a	b
--> *A	A	A

Step 7:- Optimized DFA Transition Diagram